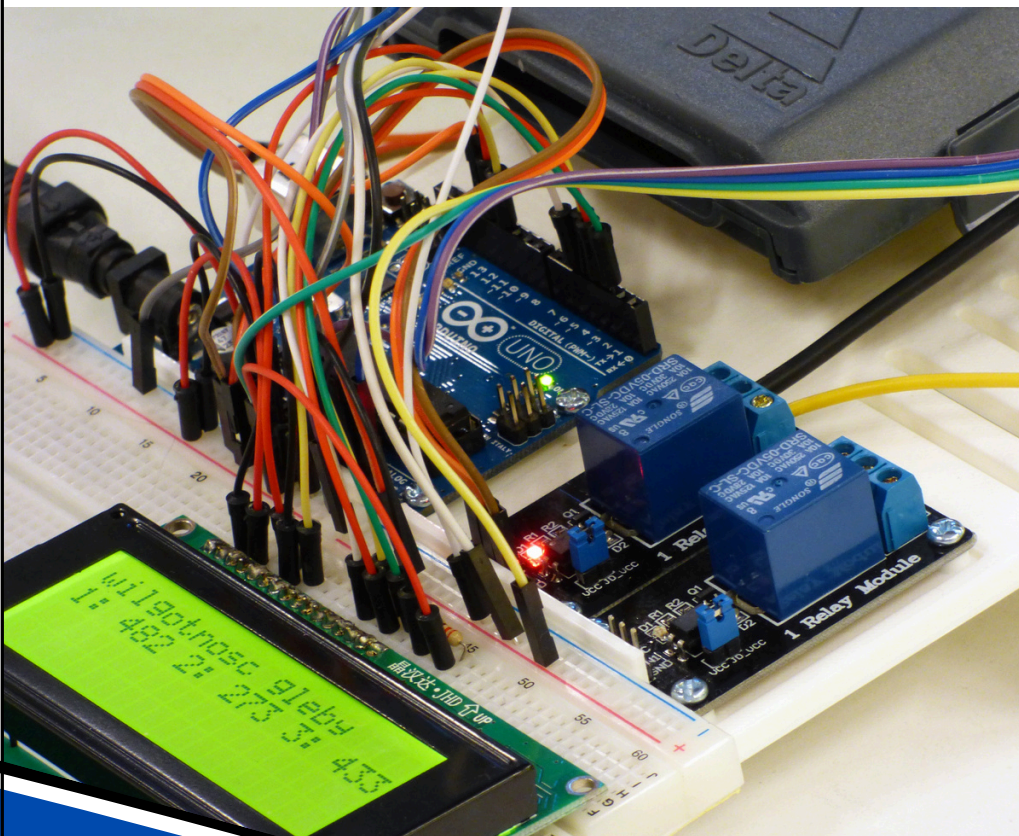


**BEGINNER TO ADVANCE
PROJECTS!**

ARDUINO TOP 75+ PROJECTS: CODE & CIRCUITS E-BOOK

Your Ultimate Guide to Building, Coding, and Exploring.



contact us
@mr_circuits007
contactmrcircuits@gmail.com

"Arduino TOP 75+ Projects: Code & Circuits"

Description:

Dive into the world of Arduino with "Arduino TOP 75+ Projects: Code & Circuits," your ultimate guide to mastering the art of electronics through hands-on projects. Whether you're a beginner eager to learn or an experienced enthusiast looking for fresh inspiration, this comprehensive eBook has something for everyone.

Unlock the potential of Arduino with this curated collection of 75+ projects, meticulously designed to cover a wide range of applications and skill levels. From basic tutorials to advanced innovations, each project is accompanied by detailed instructions, clear circuit diagrams, and ready-to-use code snippets, ensuring a smooth and rewarding learning experience.

Embark on a journey of exploration as you discover the endless possibilities of Arduino. Start with simple LED blinkers and progress to sophisticated robotics, home automation systems, IoT devices, and more. With each project, you'll gain valuable insights into electronics, programming, and problem-solving, empowering you to unleash your creativity and bring your ideas to life.

Whether you're interested in building a weather station, controlling appliances with your smartphone, creating interactive artwork, or designing your own gadgets, "Arduino TOP 75+ Projects: Code & Circuits" has got you covered. Each project is carefully crafted to provide practical skills and real-world applications, ensuring that you not only learn the fundamentals but also develop the confidence to tackle your own projects independently.

No matter your background or experience level, this eBook serves as your companion on your Arduino journey. Beginners will appreciate the step-by-step instructions and gentle introduction to electronics and programming concepts, while seasoned makers will find plenty of challenging projects to sink their teeth into. With clear explanations and troubleshooting tips, you'll overcome obstacles with ease and emerge as a proficient Arduino enthusiast.

What sets "Arduino TOP 75+ Projects: Code & Circuits" apart is its focus on both theory and practice. Each project is accompanied by concise explanations of the underlying principles, helping you understand not just how to build it, but why it works. Whether you're interested in learning about sensors, actuators, communication protocols, or power management, you'll find everything you need to know within these pages.

In addition to being a valuable learning resource, this eBook also serves as a source of inspiration for your own projects. Use the provided code as a starting point and customize it to suit your needs. Experiment with different components, sensors, and algorithms to create unique solutions tailored to your specific requirements. With Arduino, the only limit is your imagination!

So why wait? Join the millions of makers around the world who have embraced Arduino as their go-to platform for creativity and innovation. Whether you're a hobbyist, a student, an educator, or a professional, "Arduino TOP 75+ Projects: Code & Circuits" is your passport to a world of endless possibilities. Get ready to embark on an exciting journey of discovery and empowerment with Arduino!

• Projects List & Name

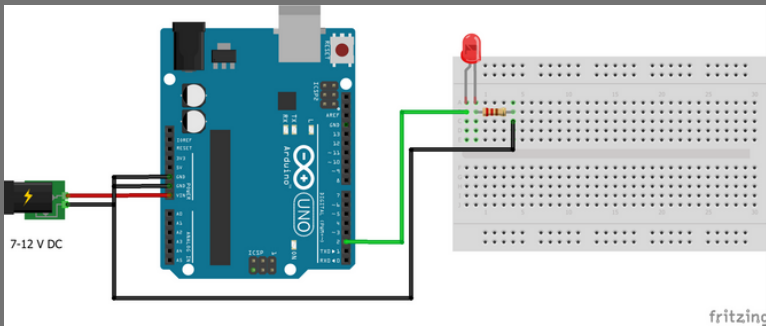
- 1.Led Blink
- 2.BLDC Motor Speed Control
- 3.Flame Sensor Projects
- 4.Flame Sensor Projects
- 5.IR Recevier Projects
- 6.Joystick Servo control
- 7.L298N Motor Driver
- 8.LCD_Display
- 9.LDR Sensor
- 10.MLX90614 With Buzzer
- 11.MLX90614 With Buzzer LCD
- 12.PIR Sensor Projects
- 13.Water Sensor
- 14.Sound Sensor
- 15.Stepper Motor
- 16.Tilt Sensor
- 17.Ultrasonic Sensor And Buzzer
- 18.Vibration Sensor
- 19Automatic Plant Watering System
- 20.Balanced Robot
- 21.Fingerprint Lock System
- 22.Human Following Robot
- 23.IR Remote Car
- 24.Line Following Robot
- 25.WiFi Car
- 26.Bluetooth control car
- 27.Digital clock
- 28.Distance Measurement using Ultrasonic sensor
- 29.Object Avoding Robot
- 30.RFID Door Lock System
- 31.Smart Dustbin
- 32.Soil Moisture Sensor
- 33.Arduino Car Parking System
- 34.Arduino Voice Control Home Automation
- 35.Arduino Text Scrolling Message Display
- 36.Arduino Digital Weight Scale
- 37.Arduino Touch Sensor
- 38.Arduino ECG Heart Rate Monitor AD8232
- 39.Arduino Keypad Solenoid Lock
- 40.Arduino Voltmeter OLED Display
- 41.Arduino Rotary Encoder controlled LEDs
- 42.Arduino OLED Stopwatch
- 43.Arduino BT Door Lock Control
- 44.Arduino LCD Notice Board
- 45.Arduino MPU6050 Digital Spirit Level Measuring Device
- 46.Arduino Secret Knock Pattern Door Lock
- 47.Arduino Home Automation using Bluetooth
- 48.Arduino Shake DICE OLED Display
- 49.Arduino Timer Control Relay
- 50.Arduino control Servo with MPU6050
- 51.Arduino RFID Door Lock System with LCD Display + 25 MORE PROJECTS

• Led Blink

• Components :

1. ARDUINO UNO
2. 5MM LED
3. 220 OHM RESISTOR
4. BREADBOARD

• Circuit Diagram :



• Code :

```
// the setup function runs once when you press reset or power the board
void setup()
{
  // initialize digital pin D2 as an output.
  pinMode(2, OUTPUT);
  Serial.begin(9600);
}

// the loop function runs over and over again forever
void loop()
{
  digitalWrite(2, HIGH);    // turn the LED on (HIGH is the voltage level)
  delay(1000);              // wait for a second
  digitalWrite(2, LOW);     // turn the LED off by making the voltage LOW
  delay(1000);              // wait for a second

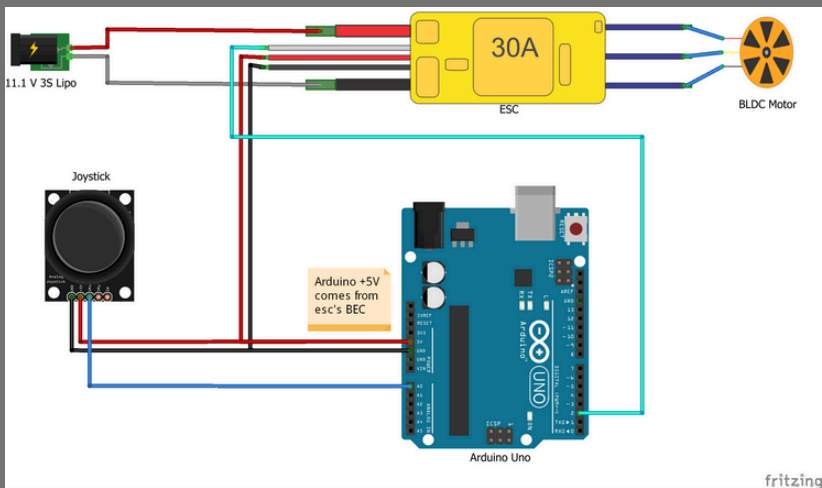
  Serial.println("Mr_circuits007");
}
```

• BLDC Motor Speed Control

• Components :

1. ARDUINO UNO
2. 30A ESC
3. BLDC MOTOR
4. JAYSTICK

• Circuit Diagram :



• Code :

```
#include<Servo.h>

#define ESC_PIN 2

Servo esc;

void setup()
{
    esc.attach(ESC_PIN, 1000, 2000);
    esc.write(0);
    delay(2000);
}

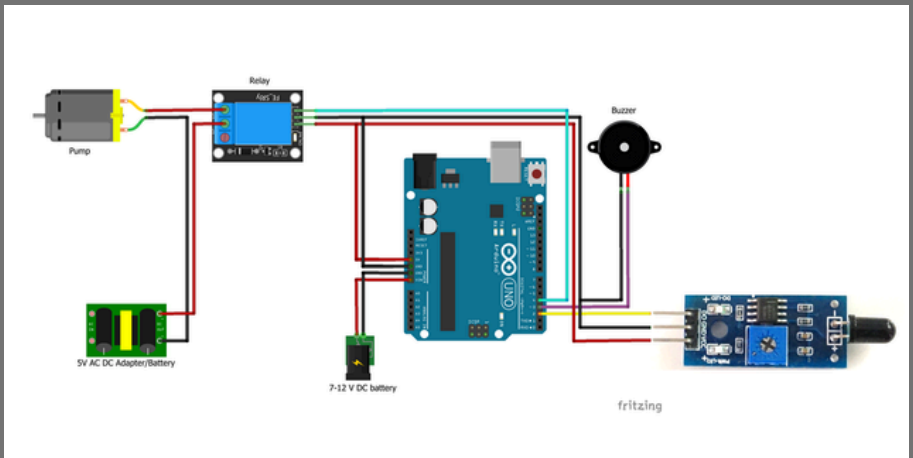
void loop()
{
    int joystickValue = analogRead(A0);
    joystickValue = constrain(joystickValue, 550, 1023);
    int mmotorSpeed = map(joystickValue, 550, 1023, 0, 180);
    esc.write(mmotorSpeed);
}
```

- **Flame Sensor Projects**

- **Components :**

1. ARDUINO UNO
2. RELAY MODULE
3. BUZZER
4. FLAME SENSOR
5. DC MOTOR

- **Circuit Diagram :**



- **Code :**

```
#define SENSOR_PIN 2
#define BUZZER_PIN 3
#define RELAY_PIN 4
#define SPRINKLER_START_DELAY 5000 //5 seconds
#define SPRINKLER_ON_TIME 3000 //3 seconds Sprinkler on time

unsigned long previousTime = millis();

void setup()
{
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(SENSOR_PIN, INPUT);
}

void loop()
{
  //If there is fire then the sensor value will be LOW else the value will be HIGH
  int sensorValue = digitalRead(SENSOR_PIN);

  //There is fire
  if (sensorValue == LOW)
  {
    analogWrite(BUZZER_PIN, 50);

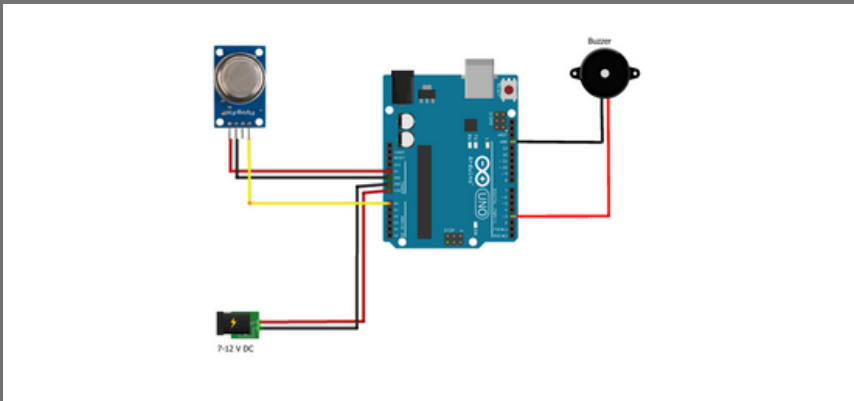
    if (millis() - previousTime > SPRINKLER_START_DELAY)
    {
      digitalWrite(RELAY_PIN, LOW);
      delay(SPRINKLER_ON_TIME);
    }
  }
  else
  {
    analogWrite(BUZZER_PIN, 0);
    digitalWrite(RELAY_PIN, HIGH);
    previousTime = millis();
  }
}
```

• Flame Sensor Projects

• Components :

1. ARDUINO UNO
2. SMOKE SENSOR
3. BUZZER

• Circuit Diagram :



• Code :

```
#define BUZZER_PIN 3

void setup()
{
  pinMode(BUZZER_PIN, OUTPUT);
  //Serial.begin(9600);
}

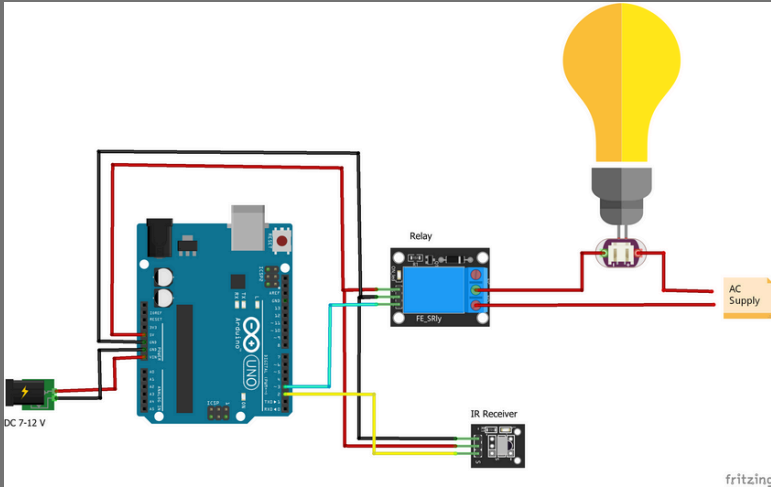
void loop()
{
  int sensorValue = analogRead(A0);
  //Serial.println(sensorValue);
  if (sensorValue > 200)
  {
    analogWrite(BUZZER_PIN, 50);
  }
  else
  {
    analogWrite(BUZZER_PIN, 0);
  }
  delay(1000);
}
```

• IR Receiver Projects

• Components :

1. ARDUINO UNO
2. IR RECEIVER
3. RELAY MODULE

• Circuit Diagram :



• Main Code :

```
#include <IRremote.h>
#include <IRremoteInt.h>
#define irPin 2
IRrecv irrecv(irPin);
decode_results results;
#define r1 3
int relay1 = LOW;

void setup()
{
  irrecv.enableIRIn();
  pinMode(r1, OUTPUT);
}

void loop() {
  if (irrecv.decode(&results)) {
    switch (results.value) {
      case 4039382595:
        relay1 = ~ relay1;
        digitalWrite(r1, relay1);
        delay(250);
        break;
    }
  }

  irrecv.resume();
}
```

• IR_Remote Get Code :

```
#include <IRremote.h>
#define RECV_PIN 2

IRrecv irrecv(RECV_PIN);
decode_results results;

void setup()
{
  Serial.begin(9600);
  Serial.println("Enabling IRin");
  irrecv.enableIRIn(); // Start the receiver
  Serial.println("Enabled IRin");
}

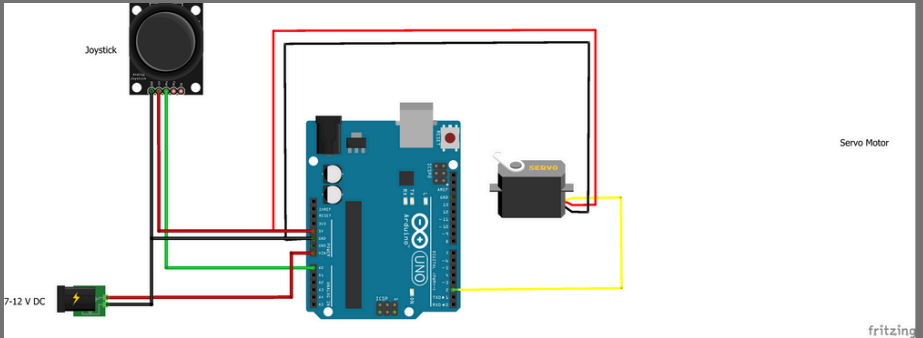
void loop()
{
  if (irrecv.decode(&results))
  {
    Serial.println(results.value, HEX);
    irrecv.resume(); // Receive the next value
  }
  delay(100);
}
```

• Joystick Servo control

• Components :

1. ARDUINO UNO
2. SERVO MOTOR
3. JAYSTICK

• Circuit Diagram :



• Main Code :

```
#include<Servo.h>

#define SERVO_PIN 2
Servo myServo;

void setup()
{
    myServo.attach(SERVO_PIN);
}

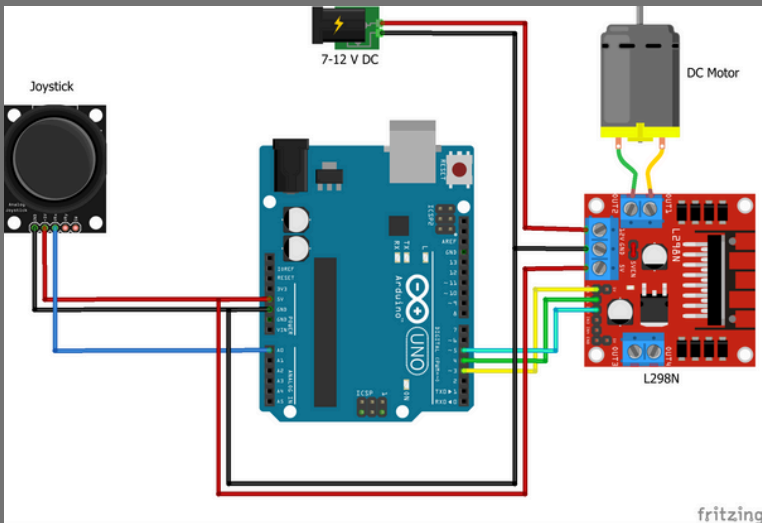
void loop()
{
    int joystickValue = analogRead(A0);
    int output = map(joystickValue, 0, 1023, 0, 180);
    myServo.write(output);
    delay(10);
}
```

- **L298N Motor Driver**

- **Components :**

1. ARDUINO UNO
2. L298D MOTOR DRIVER
3. JAYSTICK
4. DC MOTOR

- **Circuit Diagram :**



- **Main Code :**

```
int enableAPin = 3;
int in1Pin = 4;
int in2Pin = 5;

void setup()
{
  pinMode(enableAPin, OUTPUT);
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
}

void loop()
{
  int motorPWMSpeed = 0;
  int joystickValue = analogRead(A0);

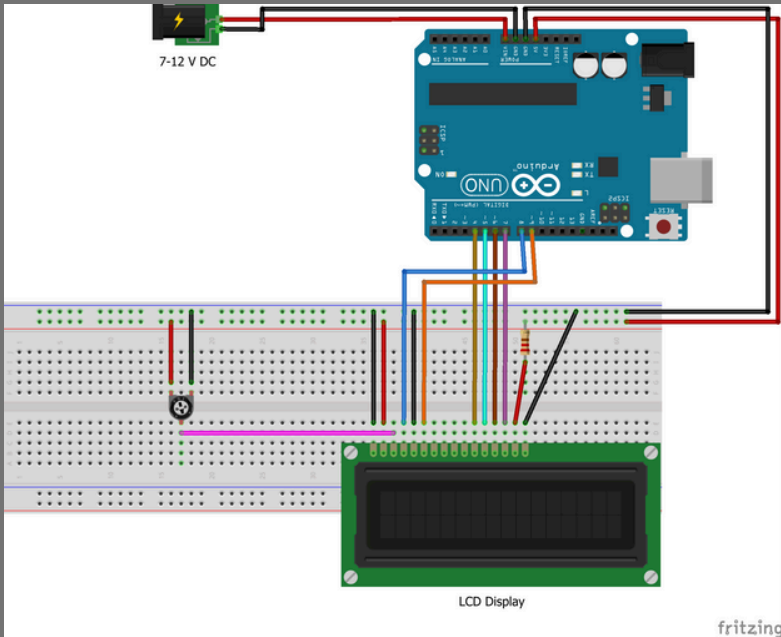
  if (joystickValue >= 530)          //This will move motor in forward direction
  {
    motorPWMSpeed = map(joystickValue, 530, 1023, 0, 255);
    digitalWrite(in1Pin, HIGH);
    digitalWrite(in2Pin, LOW);
    analogWrite(enableAPin, motorPWMSpeed);
  }
  else if (joystickValue <= 490)    //This will move motor in reverse direction
  {
    motorPWMSpeed = map(joystickValue, 490, 0, 0, 255);
    digitalWrite(in1Pin, LOW);
    digitalWrite(in2Pin, HIGH);
    analogWrite(enableAPin, motorPWMSpeed);
  }
  else                             //Stop the motor
  {
    digitalWrite(in1Pin, LOW);
    digitalWrite(in2Pin, LOW);
  }
}
```

• LCD_Display

• Components :

1. ARDUINO UNO
2. 16X2 DISPLAY
3. 10K PRESAT
4. 220 OHM
5. BREADBOARD

• Circuit Diagram :



• Main Code :

```
#include <LiquidCrystal.h>

const int rs = 8, en = 9, d4 = 4, d5 = 5, d6 = 6, d7 = 7;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup()
{
  lcd.begin(16, 2);
  lcd.clear();

  lcd.setCursor(0, 0);    //first column, First row
  lcd.print("Hello All !!");
  lcd.setCursor(0, 1);    //First column, Second row
  lcd.print("I am Ujwal");
}

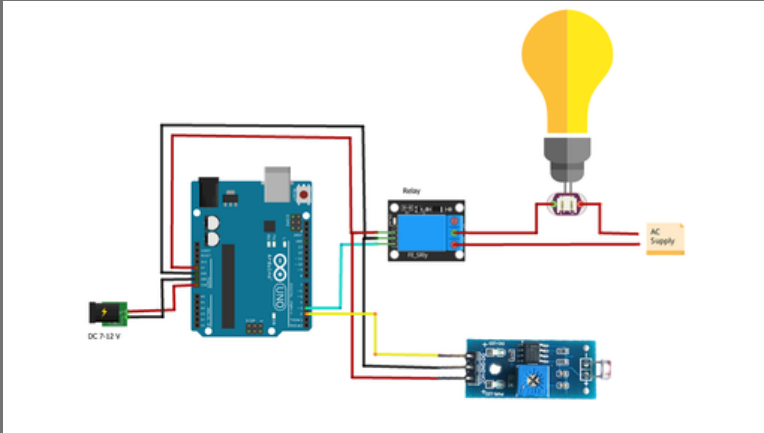
void loop()
{
  // Turn off the display:
  lcd.noDisplay();
  delay(1000);
  // Turn on the display:
  lcd.display();
  delay(1000);
}
```

• LDR Sensor

• Components :

1. ARDUINO UNO
2. LDR
3. RELAY MODULE

• Circuit Diagram :



• Main Code :

```
#define SENSOR_PIN 2
#define RELAY_PIN 3

void setup()
{
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(SENSOR_PIN, INPUT);
  //Serial.begin(9600);
}

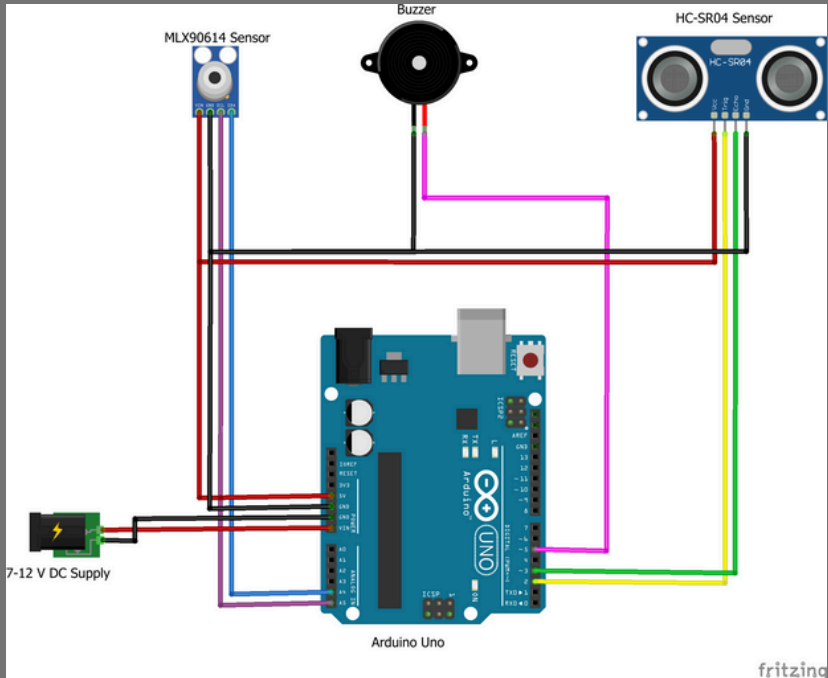
void loop()
{
  int sensorValue = digitalRead(SENSOR_PIN);
  //Serial.println(sensorValue);
  if (sensorValue == HIGH)
  {
    digitalWrite(RELAY_PIN, LOW);
  }
  else
  {
    digitalWrite(RELAY_PIN, HIGH);
  }
  delay(1000);
}
```

• MLX90614 With Buzzer

• Components :

1. ARDUINO UNO
2. MLX90614 SENSOR
3. ULTRASONIC SENSOR
4. BUZZER

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
#include <NewPing.h>
#include <Adafruit_MLX90614.h>

#define BUZZER_PIN 5
#define CUT_OFF_TEMPERATURE 30 //This is temperature in degreeC
#define DISTANCE_TO_CHECK 10 //This is in cm

NewPing ultrasonicSensor(2,3,400);
Adafruit_MLX90614 mlx = Adafruit_MLX90614();

void setup()
{
    analogWrite(BUZZER_PIN, 0);
    mlx.begin();
}

void tempHighTone()
{
    for (int i = 0; i < 10; i++)
    {
        analogWrite(BUZZER_PIN,100);
        delay(100);
        analogWrite(BUZZER_PIN,0);
        delay(100);
    }
}

void tempOkTone()
{
    analogWrite(BUZZER_PIN, 100);
    delay(2000);
    analogWrite(BUZZER_PIN, 0);
}

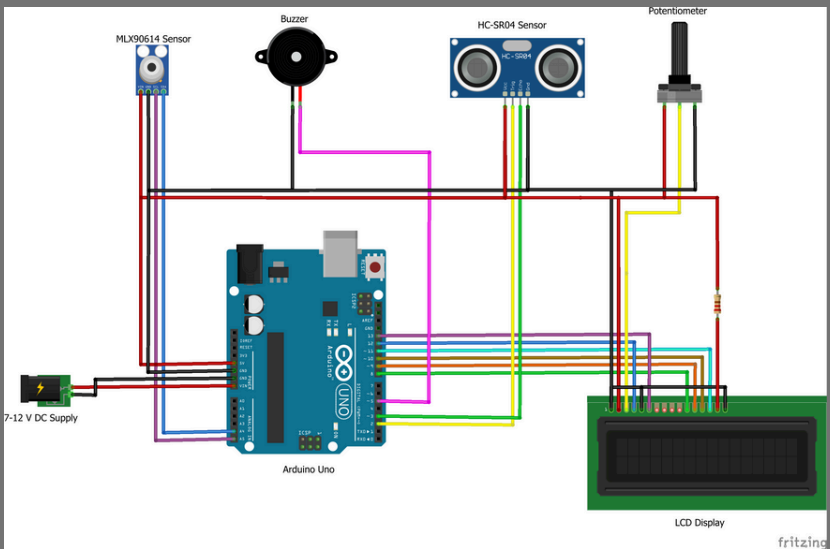
void loop()
{
    int distance = ultrasonicSensor.ping_cm();
    if (distance > 0 && distance < DISTANCE_TO_CHECK)
    {
        delay(2000);
        float temperature = mlx.readObjectTempC();
        distance = ultrasonicSensor.ping_cm();
        if (distance > 0 && distance < DISTANCE_TO_CHECK && temperature <=
CUT_OFF_TEMPERATURE)
        {
            tempOkTone();
            delay(2000);
        }
        else if ( !(distance > 0 && distance < DISTANCE_TO_CHECK))
        {
        }
        else
        {
            tempHighTone();
            delay(2000);
        }
    }
}
```

• MLX90614 With Buzzer LCD

• Components :

1. ARDUINO UNO
2. 16X2 DISPLAY
3. MLX90614 SENSOR
4. ULTRASONIC SENSOR
5. BUZZER
6. 220 OHM
7. 10K PRESAT

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
#include <NewPing.h>
#include <Adafruit_MLX90614.h>
#include <LiquidCrystal.h>

#define BUZZER_PIN 5
#define CUT_OFF_TEMPERATURE 25 //This is temperature in degreeC
#define DISTANCE_TO_CHECK 10 //This is in cm

NewPing ultrasonicSensor(2,3,400); //Ultrasonic sensor pins
Adafruit_MLX90614 mlx = Adafruit_MLX90614();

const int rs = 8, en = 9, d4 = 10, d5 = 11, d6 = 12, d7 = 13;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup()
{
  lcd.begin(16, 2);
  lcd.clear();

  analogWrite(BUZZER_PIN, 0);
  mlx.begin();
}

void tempHighTone()
{
  for (int i = 0; i < 10; i++)
  {
    analogWrite(BUZZER_PIN,100);
    delay(100);
    analogWrite(BUZZER_PIN,0);
    delay(100);
  }
}

void tempOkTone()
{
  analogWrite(BUZZER_PIN, 100);
  delay(2000);
  analogWrite(BUZZER_PIN, 0);
}

void loop()
{
  String displayString;
  lcd.setCursor(0, 0);
  lcd.print("Check Temp");

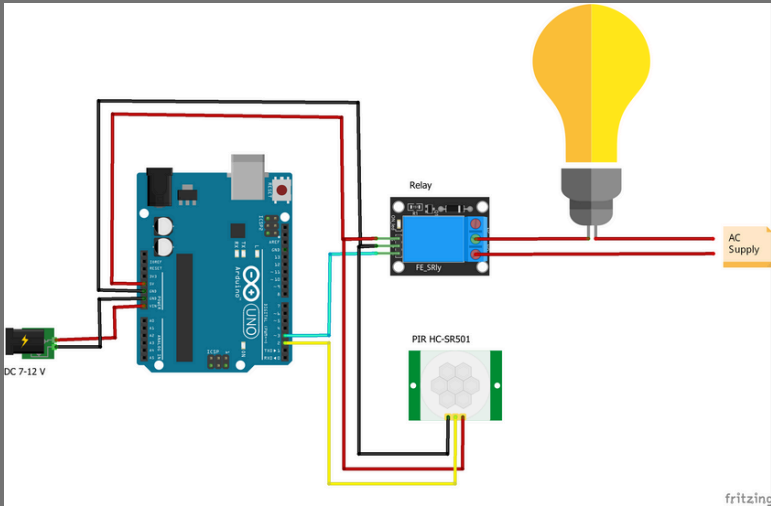
  int distance = ultrasonicSensor.ping_cm();
  if (distance > 0 && distance < DISTANCE_TO_CHECK)
  {
    delay(2000);
    float temperature = mlx.readObjectTempC();
    distance = ultrasonicSensor.ping_cm();
    if (distance > 0 && distance < DISTANCE_TO_CHECK && temperature <= CUT_OFF_TEMPERATURE)
    {
      displayString = displayString + "Temp OK : " + int(temperature);
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print(displayString);   lcd.print((char)223);   lcd.print("C");
      tempOkTone();
      delay(2000);
      lcd.clear();
    }
    else if ( !(distance > 0 && distance < DISTANCE_TO_CHECK))
    {
    }
    else
    {
      displayString = displayString + "Temp HIGH : " + int(temperature);
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print(displayString); lcd.print((char)223); lcd.print("C");
      tempHighTone();
      delay(2000);
      lcd.clear();
    }
  }
}
```

• PIR Sensor Projects

• Components :

1. ARDUINO UNO
2. PIR SENSOR
3. RELAY MODULE

• Circuit Diagram :



• Main Code :

```
#define SENSOR_PIN 2
#define RELAY_PIN 3

void setup()
{
    pinMode(RELAY_PIN, OUTPUT);
    pinMode(SENSOR_PIN, INPUT);
}

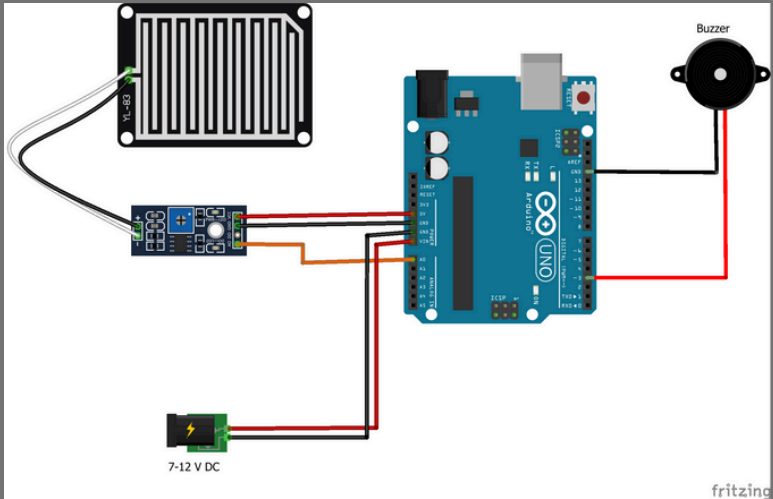
void loop()
{
    int sensorValue = digitalRead(SENSOR_PIN);
    if (sensorValue == HIGH)
    {
        digitalWrite(RELAY_PIN, LOW);
    }
    else
    {
        digitalWrite(RELAY_PIN, HIGH);
    }
}
```

• Water Sensor

• Components :

1. ARDUINO UNO
2. WATER SENSOR
3. BUZZER

• Circuit Diagram :



• Main Code :

```
#define BUZZER_PIN 3

void setup()
{
  pinMode(BUZZER_PIN, OUTPUT);
  Serial.begin(9600);
}

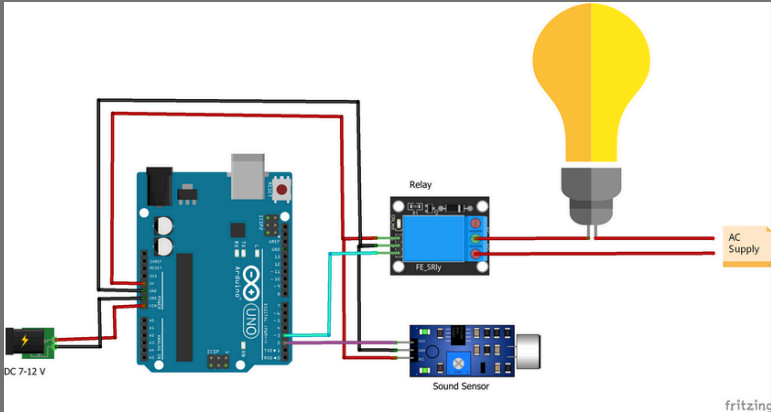
void loop()
{
  int sensorValue = analogRead(A0);
  Serial.println(sensorValue);
  if (sensorValue < 1000)
  {
    analogWrite(BUZZER_PIN, 50);
  }
  else
  {
    analogWrite(BUZZER_PIN, 0);
  }
  delay(1000);
}
```

• Sound Sensor

• Components :

1. ARDUINO UNO
2. SOUND SENSOR
3. RELAY MODULE

• Circuit Diagram :



• Main Code :

```
#define SENSOR_PIN 2
#define RELAY_PIN 3

int previousValue = HIGH;
int clapCount = 0;
unsigned long previousTime = millis();
bool relayOutPut = HIGH;

void setup()
{
  pinMode(SENSOR_PIN , INPUT);
  pinMode(RELAY_PIN, OUTPUT);

  //Turn off relay . Relay is LOW level triggered relay
  digitalWrite(RELAY_PIN, HIGH);
}

void loop()
{
  int currentValue = digitalRead(SENSOR_PIN);
  if (previousValue == HIGH && currentValue == LOW)
  {
    if (clapCount == 1 && millis() - previousTime >= 500)
    {
      clapCount = 0;
    }

    if (clapCount == 0)
    {
      previousTime = millis();
    }
    clapCount++;

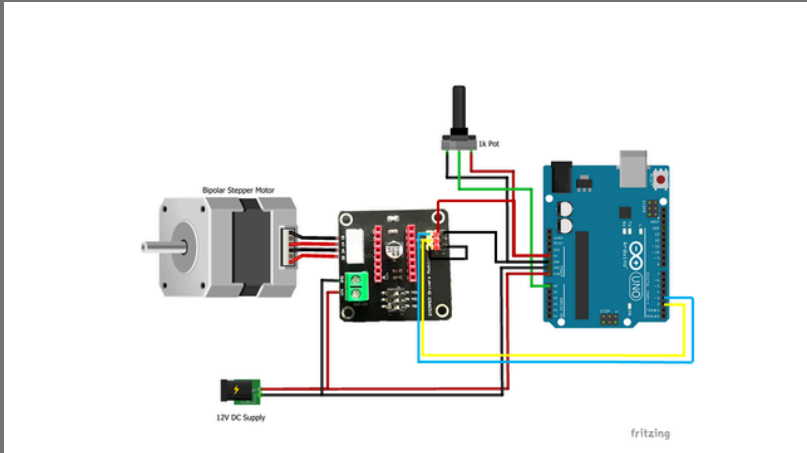
    if (clapCount == 2)
    {
      relayOutPut = !relayOutPut;
      digitalWrite(RELAY_PIN, relayOutPut);
      clapCount = 0;
    }
    delay(200);
  }
  previousValue = currentValue;
}
```

• Stepper Motor

• Components :

1. ARDUINO UNO
2. STEPPER MOTOR
3. STEPPER MOTOR DRIVER
4. 1K POT

• Circuit Diagram :



• Main Code :

```
#include <AccelStepper.h>

AccelStepper stepper(AccelStepper::DRIVER, 2, 3);

void setup()
{
    stepper.setMaxSpeed(4000);
}

void loop()
{
    int potValue = analogRead(A0);
    if (potValue < 100 ) potValue = 0;
    int stepperMotorSpeed = map(potValue, 0, 1023, 0, 4000);

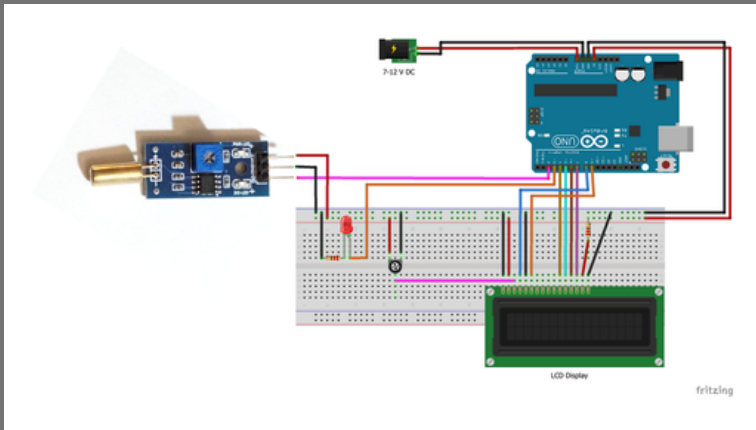
    stepper.setSpeed(stepperMotorSpeed);
    stepper.runSpeed();
}
```

• Tilt Sensor

• Components :

1. ARDUINO UNO
2. TILT SENSOR
3. 16X2 DISPLAY
4. 5MM LED
5. 10K POT
6. BREADBOARD

• Circuit Diagram :



• Main Code :

```
#include <LiquidCrystal.h>

#define SENSOR_PIN 2
#define LED_PIN 3

const int rs = 8, en = 9, d4 = 4, d5 = 5, d6 = 6, d7 = 7;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup()
{
  pinMode(LED_PIN, OUTPUT);
  lcd.begin(16, 2);
  lcd.clear();

  lcd.setCursor(0, 0); //first column, First row
  lcd.print("Test"); //Dummy print.
}

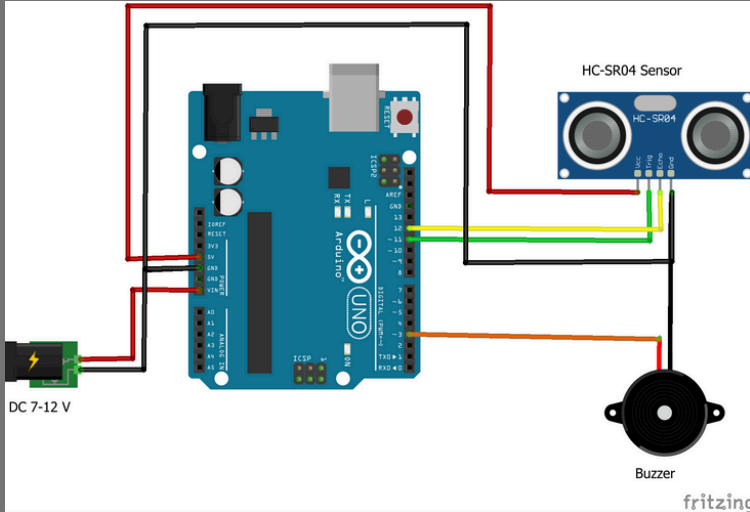
void loop()
{
  lcd.setCursor(0, 0); //first column, First row
  lcd.clear();
  if (digitalRead(SENSOR_PIN) == LOW)
  {
    lcd.print("Tilt Detected");
    digitalWrite(LED_PIN, HIGH);
  }
  else
  {
    lcd.print("Not tilted");
    digitalWrite(LED_PIN, LOW);
  }
  delay(500);
}
```

• Ultrasonic Sensor And Buzzer

• Components :

1. ARDUINO UNO
2. ULTRASONIC SENSOR
3. BUZZER

• Circuit Diagram :



• Main Code :

```
#include <NewPing.h>

#define BUZZER_PIN 3
NewPing mySensor(11,12,400);

void setup()
{
  pinMode(BUZZER_PIN, OUTPUT);
}

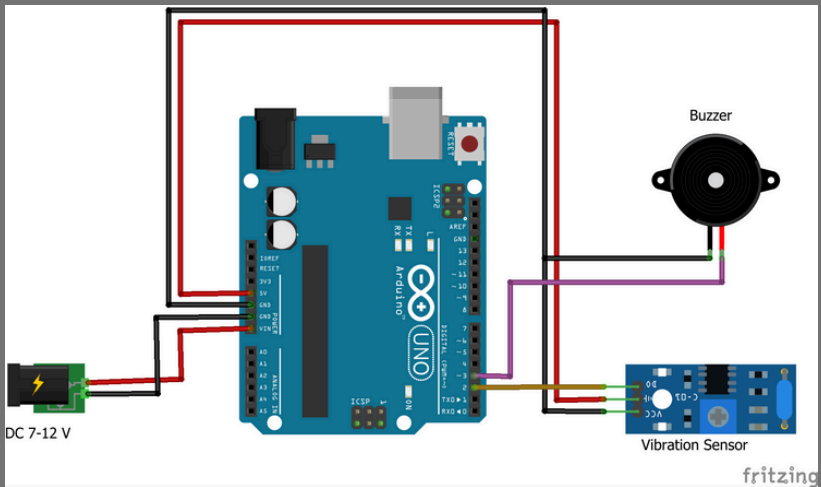
void loop()
{
  int distance = mySensor.ping_cm();
  if (distance > 0 && distance < 50)
  {
    analogWrite(BUZZER_PIN, 50);
  }
  else
  {
    analogWrite(BUZZER_PIN, 0);
  }
  delay(50);
}
```

• Vibration Sensor

• Components :

1. ARDUINO UNO
2. VIBRATION SENSOR
3. BUZZER

• Circuit Diagram :



• Main Code :

```
#define SENSOR_PIN 2
#define BUZZER_PIN 3

void setup()
{
    pinMode(SENSOR_PIN, INPUT);
    //Serial.begin(9600);
}

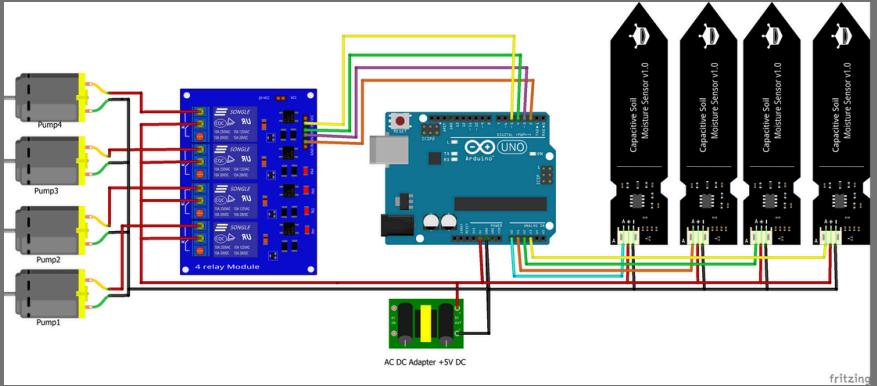
void loop()
{
    //If there is no vibration then the sensor value will be 0 else the value will be 1
    int sensorValue = digitalRead(SENSOR_PIN);
    //Serial.println(sensorValue);
    //There is fire
    if (sensorValue == LOW)
    {
        analogWrite(BUZZER_PIN, 0);
    }
    else
    {
        //If vibration then turn on the buzzer
        analogWrite(BUZZER_PIN, 50);
    }
    //You can add delay between reading
    delay(100);
}
```

• Automatic Plant Watering System

• Components :

1. ARDUINO UNO
2. SOIL SENSOR
3. 4CHANNEL RELAY MODULE
4. 4X DC WATER PUMP

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
#define SENSOR_READ_DELAY 86400000
#define FULL_MOISTURE_READING 290
#define NO_MOISTURE_READING 595
#define CUT_OFF_MOISTURE_PERCENTAGE 50
#define PUMP1_WATERING_TIME 2000
#define PUMP2_WATERING_TIME 2000
#define PUMP3_WATERING_TIME 2000
#define PUMP4_WATERING_TIME 2000

int pumpPin1=2;
int pumpPin2=3;
int pumpPin3=4;
int pumpPin4=5;

int inputSensorPin1=A0;
int inputSensorPin2=A1;
int inputSensorPin3=A2;
int inputSensorPin4=A3;

void setup()
{
    // put your setup code here, to run once:
    pinMode(pumpPin1,OUTPUT);
    pinMode(pumpPin2,OUTPUT);
    pinMode(pumpPin3,OUTPUT);
    pinMode(pumpPin4,OUTPUT);

    digitalWrite(pumpPin1,HIGH);
    digitalWrite(pumpPin2,HIGH);
    digitalWrite(pumpPin3,HIGH);
    digitalWrite(pumpPin4,HIGH);
}

void getMoisturePercentageAndWaterIt(int sensorValue, int outPinNo, int wateringTime)
{
    sensorValue = constrain(sensorValue ,FULL_MOISTURE_READING,
    NO_MOISTURE_READING);
    int moisturePercentage = map(sensorValue, FULL_MOISTURE_READING,
    NO_MOISTURE_READING, 100, 0);

    if (moisturePercentage < CUT_OFF_MOISTURE_PERCENTAGE)
    {
        digitalWrite(outPinNo, LOW);
        delay(wateringTime);
        digitalWrite(outPinNo, HIGH);
    }
}

void loop()
{
    // put your main code here, to run repeatedly:
    int inputSensorPin1Value = analogRead(inputSensorPin1);
    getMoisturePercentageAndWaterIt(inputSensorPin1Value, pumpPin1,
    PUMP1_WATERING_TIME);

    int inputSensorPin2Value = analogRead(inputSensorPin2);
    getMoisturePercentageAndWaterIt(inputSensorPin2Value, pumpPin2,
    PUMP2_WATERING_TIME);

    int inputSensorPin3Value = analogRead(inputSensorPin3);
    getMoisturePercentageAndWaterIt(inputSensorPin3Value, pumpPin3,
    PUMP3_WATERING_TIME);

    int inputSensorPin4Value = analogRead(inputSensorPin4);
    getMoisturePercentageAndWaterIt(inputSensorPin4Value, pumpPin4,
    PUMP4_WATERING_TIME);

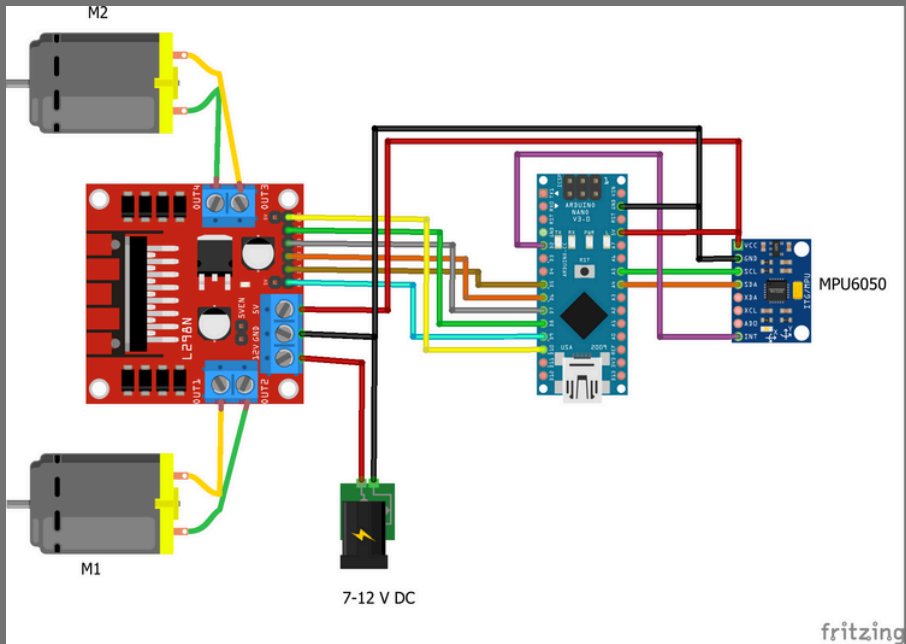
    delay(SENSOR_READ_DELAY);
}
```

- **Balanced Robot**

- **Components :**

1. ARDUINO NANO
2. L298D MOTOR DRIVER
3. MPU6050
4. DC MOTOR

- **Circuit Diagram :**



- **Main Code :**

• Main Code :

```
##define PRINT_DEBUG_BUILD //This is to print the mpu data on serial monitor to debug
//PID library
#include <PID_v1.h>

//These are needed for MPU
#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"
// Arduino Wire library is required if I2Cdev I2CDEV_ARDUINO_WIRE implementation
// is used in I2Cdev.h
#if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
#include "Wire.h"
#else
#endif

// class default I2C address is 0x68
MPU6050 mpu;

#define INTERRUPT_PIN 2 // use pin 2 on Arduino Uno & most boards

// MPU control/status vars
bool dmpReady = false; // set true if DMP init was successful
uint8_t mpuIntStatus; // holds actual interrupt status byte from MPU
uint8_t devStatus; // return status after each device operation (0 = success, != 0 = error)
uint16_t packetSize; // expected DMP packet size (default is 42 bytes)
uint16_t fifoCount; // count of all bytes currently in FIFO
uint8_t fifoBuffer[64]; // FIFO storage buffer

// orientation/motion vars
Quaternion q; // [w, x, y, z] quaternion container
VectorFloat gravity; // [x, y, z] gravity vector
float ypr[3]; // [yaw, pitch, roll] yaw/pitch/roll container and gravity vector
VectorInt16 gy; // [x, y, z] gyro sensor measurements

// INTERRUPT DETECTION ROUTINE

volatile bool mpuInterrupt = false; // indicates whether MPU interrupt pin has gone high
void dataReady() {
    mpuInterrupt = true;
}

#define PID_MIN_LIMIT -255 //Min limit for PID
#define PID_MAX_LIMIT 255 //Max limit for PID
#define PID_SAMPLE_TIME_IN_MILLI 10 //This is PID sample time in milliseconds

//The pitch angle given by MPU6050 when robot is vertical and MPU6050 is horizontal is 0 in ideal case.
//However in real case its slightly off and we need add some correction to keep robot vertical.
//This is the angle correction to keep our robot stand vertically. Sometimes robot moves in one direction so we need to adjust this.
#define SETPOINT_PITCH_ANGLE_OFFSET -2.2

#define MIN_ABSOLUTE_SPEED 0 //Min motor speed

double setpointPitchAngle = SETPOINT_PITCH_ANGLE_OFFSET;
double pitchGyroAngle = 0;
double pitchPIDOutput = 0;

double setpointYawRate = 0;
double yawGyroRate = 0;
double yawPIDOutput = 0;

#define PID_PITCH_KP 10
#define PID_PITCH_KI 80
#define PID_PITCH_KD .8

#define PID_YAW_KP 0.5
#define PID_YAW_KI 0.5
#define PID_YAW_KD 0

PID pitchPID(&pitchGyroAngle, &pitchPIDOutput, &setpointPitchAngle, PID_PITCH_KP, PID_PITCH_KI, PID_PITCH_KD, DIRECT);
PID yawPID(&yawGyroRate, &yawPIDOutput, &setpointYawRate, PID_YAW_KP, PID_YAW_KI, PID_YAW_KD, DIRECT);

int enableMotor1=9;
int motor1Pin1=5;
int motor1Pin2=6;

int motor2Pin1=7;
int motor2Pin2=8;
int enableMotor2=10;

void setupPID()
{
    pitchPID.SetOutputLimits(PID_MIN_LIMIT, PID_MAX_LIMIT);
    pitchPID.SetMode(AUTOMATIC);
    pitchPID.SetSampleTime(PID_SAMPLE_TIME_IN_MILLI);

    yawPID.SetOutputLimits(PID_MIN_LIMIT, PID_MAX_LIMIT);
    yawPID.SetMode(AUTOMATIC);
    yawPID.SetSampleTime(PID_SAMPLE_TIME_IN_MILLI);
}

void setupMotors()
{
    pinMode(enableMotor1,OUTPUT);
    pinMode(motor1Pin1,OUTPUT);
    pinMode(motor1Pin2,OUTPUT);

    pinMode(enableMotor2,OUTPUT);
    pinMode(motor2Pin1,OUTPUT);
    pinMode(motor2Pin2,OUTPUT);

    rotateMotor(0,0);
}
```

```

void setupMPU()
{
    // join I2C bus (I2Cdev library doesn't do this automatically)
    #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
        Wire.begin();
        Wire.setClock(400000); // 400kHz I2C clock. Comment this line if having compilation difficulties
    #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
        Fastwire::setup(400, true);
    #endif

    mpu.initialize();
    pinMode(INTERRUPT_PIN, INPUT);
    devStatus = mpu.dmpInitialize();

    // supply your own gyro offsets here, scaled for min sensitivity
    //      X Accel  Y Accel  Z Accel  X Gyro  Y Gyro  Z Gyro
    //OFFSETS  -1798,    263,    1124,   -261,    26,    32
    mpu.setXAccelOffset(-1798);
    mpu.setYAccelOffset(263);
    mpu.setZAccelOffset(1124);
    mpu.setXGyroOffset(-261);
    mpu.setYGyroOffset(26);
    mpu.setZGyroOffset(32);
    // make sure it worked (returns 0 if so)
    if (devStatus == 0)
    {
        // Calibration Time: generate offsets and calibrate our MPU6050
        //mpu.CalibrateAccel(6);
        //mpu.CalibrateGyro(6);
        // turn on the DMP, now that it's ready
        mpu.setDMPEnabled(true);
        mpu.IntStatus = mpu.getIntStatus();
        dmpReady = true;
        // get expected DMP packet size for later comparison
        packetSize = mpu.dmpGetFIFOPacketSize();
    }
    else
    {
        // ERROR!
    }
}

void setup()
{
    //This is to set up motors
    setupMotors();
    //This is to set up MPU6050 sensor
    setupMPU();
    //This is to set up PID
    setupPID();
}

void loop()
{
    // if programming failed, don't try to do anything
    if (!dmpReady) return;

    // read a packet from FIFO. Get the latest packet
    if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer))
    {
        mpu.dmpGetQuaternion(&q, fifoBuffer);
        mpu.dmpGetGravity(&gravity, &q);
        mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
        mpu.dmpGetGyro(&gy, fifoBuffer);

        yawGyroRate = gy.z; //rotation rate in degrees per second
        pitchGyroAngle = ypr[1] * 180/M_PI; //angle in degree

        pitchPID.Compute(true);
        yawPID.Compute(true);

        rotateMotor(pitchPIDOutput*yawPIDOutput, pitchPIDOutput-yawPIDOutput);

        #ifdef PRINT_DEBUG_BUILD
            Serial.println("The gyro before ");
            Serial.println(pitchGyroAngle);
            Serial.println("The setpoints ");
            Serial.println(setpointPitchAngle);
            Serial.println("The pid output ");
            Serial.println(pitchPIDOutput);
            delay(500);
        #endif
    }
}

void rotateMotor(int speed1, int speed2)
{
    if (speed1 < 0)
    {
        digitalWrite(motor1Pin1, LOW);
        digitalWrite(motor1Pin2, HIGH);
    }
    else if (speed1 >= 0)
    {
        digitalWrite(motor1Pin1, HIGH);
        digitalWrite(motor1Pin2, LOW);
    }

    if (speed2 < 0)
    {
        digitalWrite(motor2Pin1, LOW);
        digitalWrite(motor2Pin2, HIGH);
    }
    else if (speed2 >= 0)
    {
        digitalWrite(motor2Pin1, HIGH);
        digitalWrite(motor2Pin2, LOW);
    }

    speed1 = abs(speed1) + MIN_ABSOLUTE_SPEED;
    speed2 = abs(speed2) + MIN_ABSOLUTE_SPEED;

    speed1 = constrain(speed1, MIN_ABSOLUTE_SPEED, 255);
    speed2 = constrain(speed2, MIN_ABSOLUTE_SPEED, 255);

    analogWrite(enableMotor1, speed1);
    analogWrite(enableMotor2, speed2);
}

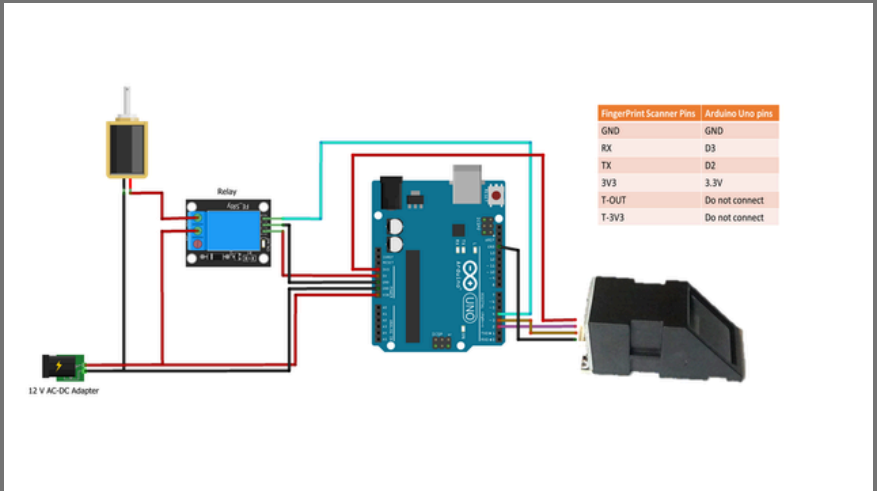
```

• Fingerprint Lock System

• Components :

1. ARDUINO UNO
2. FINGERPRINT SENSOR
3. RELAY MODULE
4. ELECTRIC DOOR LOCK

• Circuit Diagram :



• Main Code :

goto next page !

- **Main Code :**

```
#include <Adafruit_Fingerprint.h>

SoftwareSerial mySerial(2, 3);
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);

#define RELAY_PIN      4
#define ACCESS_DELAY    3000 // Keep lock unlocked for 3 seconds

void setup()
{
    // set the data rate for the sensor serial port
    finger.begin(57600);
    delay(5);
    if (finger.verifyPassword())
    {
    }
    else
    {
        while (1) { delay(1); }
    }

    pinMode(RELAY_PIN, OUTPUT);
    digitalWrite(RELAY_PIN, HIGH); //Switch off relay initially. Relay is
LOW level triggered relay so we need to write HIGH.
}

void loop()
{
    if ( getFingerPrint() != -1)
    {
        digitalWrite(RELAY_PIN, LOW);
        delay(ACCESS_DELAY);
        digitalWrite(RELAY_PIN, HIGH);
    }
    delay(50);          //Add some delay before next scan.
}

// returns -1 if failed, otherwise returns ID #
int getFingerPrint()
{
    int p = finger.getImage();
    if (p != FINGERPRINT_OK) return -1;

    p = finger.image2Tz();
    if (p != FINGERPRINT_OK) return -1;

    p = finger.fingerFastSearch();
    if (p != FINGERPRINT_OK) return -1;

    // found a match!
    return finger.fingerID;
}
```

• THIS CODE FOR FINGERPRINT ENROLL :

```
#include <Adafruit_Fingerprint.h>

SoftwareSerial mySerial(2, 3);
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);

uint8_t id;

void setup()
{
  Serial.begin(9600);
  while (!Serial);
  delay(100);

  // set the data rate for the sensor serial port
  finger.begin(57600);
  if (finger.verifyPassword())
  {
    Serial.println("Found fingerprint sensor!");
  }
  else
  {
    Serial.println("Did not find fingerprint sensor :(");
    while (1) { delay(1); }
  }

  Serial.println(F("Reading sensor parameters"));
  finger.getParameters();
  Serial.print(F("TemplateCount: ")); Serial.println(finger.templateCount);
  Serial.print(F("Status: 0x")); Serial.println(finger.status_reg, HEX);
  Serial.print(F("Sys ID: 0x")); Serial.println(finger.system_id, HEX);
  Serial.print(F("Capacity: ")); Serial.println(finger.capacity);
  Serial.print(F("Security level: ")); Serial.println(finger.security_level);
  Serial.print(F("Device address: ")); Serial.println(finger.device_addr, HEX);
  Serial.print(F("Packet len: ")); Serial.println(finger.packet_len);
  Serial.print(F("Baud rate: ")); Serial.println(finger.baud_rate);
  Serial.print(F("Total finger prints stored: ")); Serial.println(finger.templateCount);
}

uint8_t readnumber(void)
{
  uint8_t num = 0;

  while (num == 0)
  {
    while (! Serial.available());
    num = Serial.parseInt();
  }
  return num;
}

void loop()
{
  Serial.println("\n\nReady to enroll a fingerprint!");
  Serial.println("Please type in -1 to delete all the stored finger prints...");
  Serial.println("Please type in the ID # (from 1 to 127) you want to save the finger as...");
  id = readnumber();
  if (id == 0) // ID #0 not allowed, try again!
  {
    return;
  }
  if (id == uint8_t(-1))
  {
    Serial.print("Cleared all finger prints from database");
    finger.emptyDatabase();
    return;
  }

  Serial.print("Enrolling ID #");
  Serial.println(id);
  while (! getFingerprintEnroll() );
}

uint8_t getFingerprintEnroll()
{
  int p = -1;
  Serial.print("Waiting for valid finger to enroll as #"); Serial.println(id);
  while (p != FINGERPRINT_OK)
  {
    p = finger.getImage();
    switch (p)
    {
      case FINGERPRINT_OK:
        Serial.println("Image taken");
        break;
      case FINGERPRINT_NOFINGER:
        break;
      case FINGERPRINT_PACKETRECEIVEERR:
        Serial.println("Communication error");
        break;
      case FINGERPRINT_IMAGEFAIL:
        Serial.println("Imaging error");
        break;
      default:
        Serial.println("Unknown error");
        break;
    }
  }

  // OK success!

  p = finger.image2Tz(1);
  switch (p)
  {
    case FINGERPRINT_OK:
      Serial.println("Image converted");
      break;
    case FINGERPRINT_IMAGEMESS:
      Serial.println("Image too messy");
      return p;
    case FINGERPRINT_PACKETRECEIVEERR:
      Serial.println("Communication error");
      return p;
    case FINGERPRINT_FEATUREFAIL:
      Serial.println("Could not find fingerprint features");
      return p;
    case FINGERPRINT_INVALIDIMAGE:
      Serial.println("Could not find fingerprint features");
      return p;
    default:
      Serial.println("Unknown error");
      return p;
  }
}
```

```

Serial.println("Remove finger");
delay(2000);
p = 0;
while (p != FINGERPRINT_NOFINGER)
{
    p = finger.getImage();
}
Serial.print("ID "); Serial.println(id);
p = -1;
Serial.println("Place same finger again");
while (p != FINGERPRINT_OK)
{
    p = finger.getImage();
    switch (p)
    {
        case FINGERPRINT_OK:
            Serial.println("Image taken");
            break;
        case FINGERPRINT_NOFINGER:
            break;
        case FINGERPRINT_PACKETRECEIVEERR:
            Serial.println("Communication error");
            break;
        case FINGERPRINT_IMAGEFAIL:
            Serial.println("Imaging error");
            break;
        default:
            Serial.println("Unknown error");
            break;
    }
}

// OK success!

p = finger.image2Tz(2);
switch (p)
{
    case FINGERPRINT_OK:
        Serial.println("Image converted");
        break;
    case FINGERPRINT_IMAGEMESS:
        Serial.println("Image too messy");
        return p;
    case FINGERPRINT_PACKETRECEIVEERR:
        Serial.println("Communication error");
        return p;
    case FINGERPRINT_FEATUREFAIL:
        Serial.println("Could not find fingerprint features");
        return p;
    case FINGERPRINT_INVALIDIMAGE:
        Serial.println("Could not find fingerprint features");
        return p;
    default:
        Serial.println("Unknown error");
        return p;
}

// OK converted!
Serial.print("Creating model for #"); Serial.println(id);

p = finger.createModel();
if (p == FINGERPRINT_OK) {
    Serial.println("Prints matched!");
} else if (p == FINGERPRINT_PACKETRECEIVEERR) {
    Serial.println("Communication error");
    return p;
} else if (p == FINGERPRINT_ENROLLMISMATCH) {
    Serial.println("Fingerprints did not match");
    return p;
} else {
    Serial.println("Unknown error");
    return p;
}

Serial.print("ID "); Serial.println(id);
p = finger.storeModel(id);
if (p == FINGERPRINT_OK) {
    Serial.println("Stored!");
} else if (p == FINGERPRINT_PACKETRECEIVEERR) {
    Serial.println("Communication error");
    return p;
} else if (p == FINGERPRINT_BADLOCATION) {
    Serial.println("Could not store in that location");
    return p;
} else if (p == FINGERPRINT_FLASHERR) {
    Serial.println("Error writing to flash");
    return p;
} else {
    Serial.println("Unknown error");
    return p;
}

return true;
}

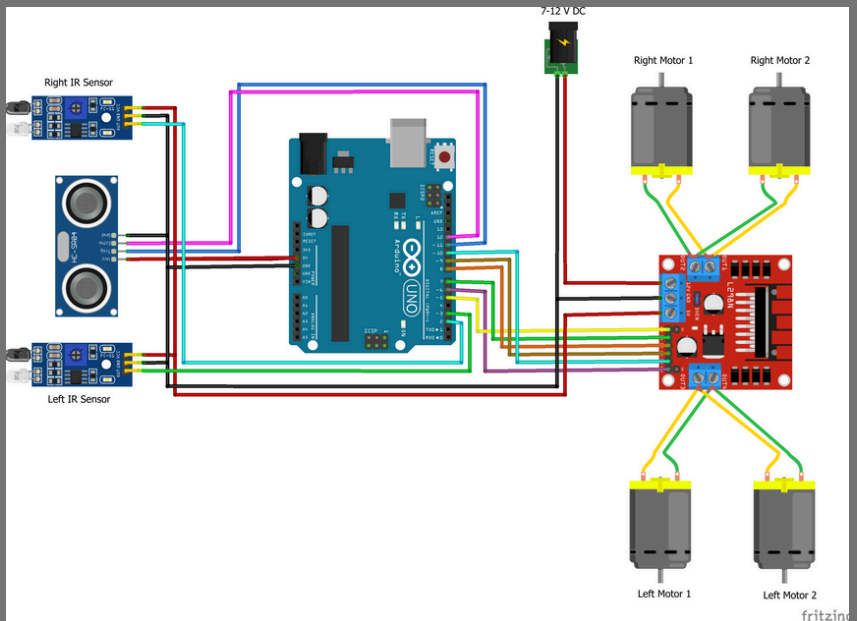
```

• Human Following Robot

• Components :

1. ARDUINO UNO
2. L298D MOTOR DRIVER
3. ULTRASONIC SENSOR
4. 2x IR PROXIMITY SENSOR
5. 4x DC MOTOR

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
#include <NewPing.h>

#define ULTRASONIC_SENSOR_TRIG 11
#define ULTRASONIC_SENSOR_ECHO 12

#define MAX_FORWARD_MOTOR_SPEED 75
#define MAX_MOTOR_TURN_SPEED_ADJUSTMENT 50

#define MIN_DISTANCE 10
#define MAX_DISTANCE 30

#define IR_SENSOR_RIGHT 2
#define IR_SENSOR_LEFT 3

//Right motor
int enableRightMotor=5;
int rightMotorPin1=7;
int rightMotorPin2=8;

//Left motor
int enableLeftMotor=6;
int leftMotorPin1=9;
int leftMotorPin2=10;

NewPing mySensor(ULTRASONIC_SENSOR_TRIG, ULTRASONIC_SENSOR_ECHO, 400);

void setup()
{
    // put your setup code here, to run once:
    pinMode(enableRightMotor, OUTPUT);
    pinMode(rightMotorPin1, OUTPUT);
    pinMode(rightMotorPin2, OUTPUT);

    pinMode(enableLeftMotor, OUTPUT);
    pinMode(leftMotorPin1, OUTPUT);
    pinMode(leftMotorPin2, OUTPUT);

    pinMode(IR_SENSOR_RIGHT, INPUT);
    pinMode(IR_SENSOR_LEFT, INPUT);
    rotateMotor(0,0);
}

void loop()
{
    int distance = mySensor.ping_cm();
    int rightIRSensorValue = digitalRead(IR_SENSOR_RIGHT);
    int leftIRSensorValue = digitalRead(IR_SENSOR_LEFT);

    //NOTE: If IR sensor detects the hand then its value will be LOW else the value will be HIGH

    //If right sensor detects hand, then turn right. We increase left motor speed and decrease the right motor speed to turn towards right
    if (rightIRSensorValue == LOW && leftIRSensorValue == HIGH )
    {
        rotateMotor(MAX_FORWARD_MOTOR_SPEED - MAX_MOTOR_TURN_SPEED_ADJUSTMENT, MAX_FORWARD_MOTOR_SPEED + MAX_MOTOR_TURN_SPEED_ADJUSTMENT );
    }
    //If left sensor detects hand, then turn left. We increase right motor speed and decrease the left motor speed to turn towards left
    else if (rightIRSensorValue == HIGH && leftIRSensorValue == LOW )
    {
        rotateMotor(MAX_FORWARD_MOTOR_SPEED + MAX_MOTOR_TURN_SPEED_ADJUSTMENT, MAX_FORWARD_MOTOR_SPEED - MAX_MOTOR_TURN_SPEED_ADJUSTMENT);
    }
    //If distance is between min and max then go straight
    else if (distance >= MIN_DISTANCE && distance <= MAX_DISTANCE)
    {
        rotateMotor(MAX_FORWARD_MOTOR_SPEED, MAX_FORWARD_MOTOR_SPEED);
    }
    //stop the motors
    else
    {
        rotateMotor(0, 0);
    }
}

void rotateMotor(int rightMotorSpeed, int leftMotorSpeed)
{
    if (rightMotorSpeed < 0)
    {
        digitalWrite(rightMotorPin1,LOW);
        digitalWrite(rightMotorPin2,HIGH);
    }
    else if (rightMotorSpeed > 0)
    {
        digitalWrite(rightMotorPin1,HIGH);
        digitalWrite(rightMotorPin2,LOW);
    }
    else
    {
        digitalWrite(rightMotorPin1,LOW);
        digitalWrite(rightMotorPin2,LOW);
    }

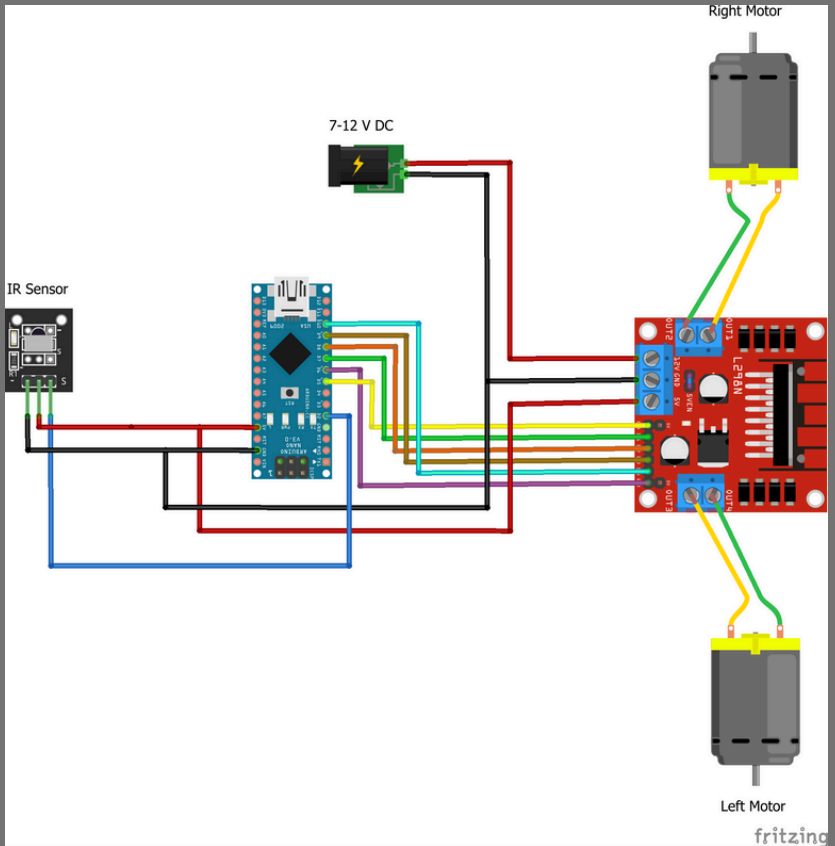
    if (leftMotorSpeed < 0)
    {
        digitalWrite(leftMotorPin1,LOW);
        digitalWrite(leftMotorPin2,HIGH);
    }
    else if (leftMotorSpeed > 0)
    {
        digitalWrite(leftMotorPin1,HIGH);
        digitalWrite(leftMotorPin2,LOW);
    }
    else
    {
        digitalWrite(leftMotorPin1,LOW);
        digitalWrite(leftMotorPin2,LOW);
    }
    analogWrite(enableRightMotor, abs(rightMotorSpeed));
    analogWrite(enableLeftMotor, abs(leftMotorSpeed));
}
```

• IR Remote Car

• Components :

1. ARDUINO NANO
2. L298D MOTOR DRIVER
3. IR RECEIVER
4. DC MOTOR

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
#include <IRremote.h>

//Right motor
int enableRightMotor=5;
int rightMotorPin1=7;
int rightMotorPin2=8;

//Left motor
int enableLeftMotor=6;
int leftMotorPin1=9;
int leftMotorPin2=10;

#define RECV_PIN 2
IRrecv irrecv(RECV_PIN);
decode_results results;

#define MAX_MOTOR_SPEED 200

void setup()
{
  // put your setup code here, to run once:
  pinMode(enableRightMotor, OUTPUT);
  pinMode(rightMotorPin1, OUTPUT);
  pinMode(rightMotorPin2, OUTPUT);

  pinMode(enableLeftMotor, OUTPUT);
  pinMode(leftMotorPin1, OUTPUT);
  pinMode(leftMotorPin2, OUTPUT);

  rotateMotor(0,0);

  irrecv.enableIRIn(); // Start the receiver
}

void rotateMotor(int rightMotorSpeed, int leftMotorSpeed)
{
  if (rightMotorSpeed < 0)
  {
    digitalWrite(rightMotorPin1, LOW);
    digitalWrite(rightMotorPin2, HIGH);
  }
  else if (rightMotorSpeed >= 0)
  {
    digitalWrite(rightMotorPin1, HIGH);
    digitalWrite(rightMotorPin2, LOW);
  }

  if (leftMotorSpeed < 0)
  {
    digitalWrite(leftMotorPin1, LOW);
    digitalWrite(leftMotorPin2, HIGH);
  }
  else if (leftMotorSpeed >= 0)
  {
    digitalWrite(leftMotorPin1, HIGH);
    digitalWrite(leftMotorPin2, LOW);
  }

  analogWrite(enableRightMotor, abs(rightMotorSpeed));
  analogWrite(enableLeftMotor, abs(leftMotorSpeed));
}

void loop()
{
  if (irrecv.decode(&results))
  {
    irrecv.resume(); // Receive the next value

    //40BD649B FORWARD
    //40BD14EB BACKWARD
    //40BD445B LEFT
    //40BD41B RIGHT
    //40BD04F STOP

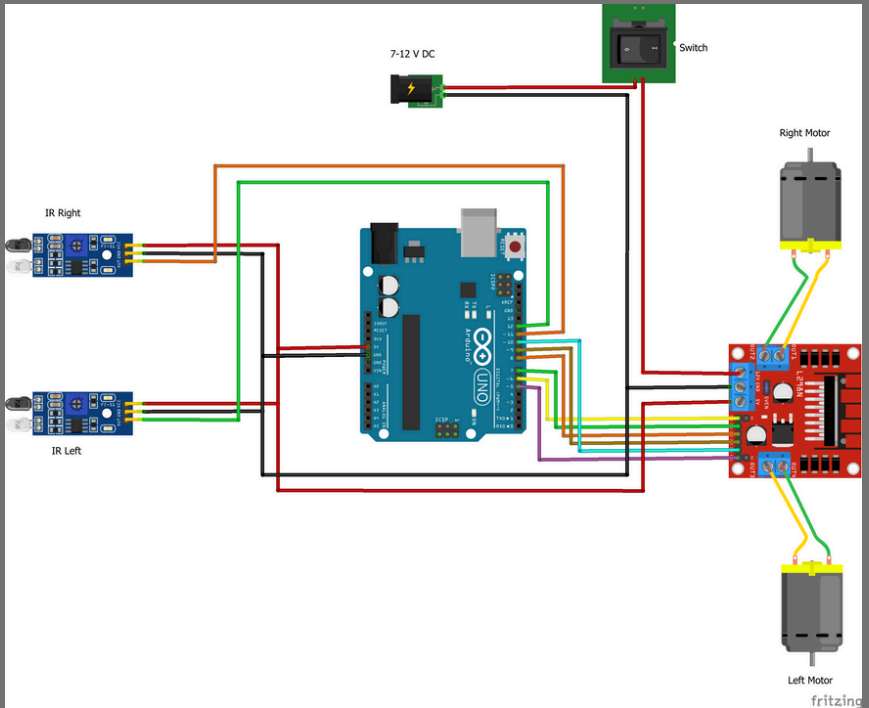
    if (results.value == 0x40BD649B)
    {
      rotateMotor(MAX_MOTOR_SPEED, MAX_MOTOR_SPEED);
    }
    else if (results.value == 0x40BD14EB)
    {
      rotateMotor(-MAX_MOTOR_SPEED, -MAX_MOTOR_SPEED);
    }
    else if (results.value == 0x40BD445B)
    {
      rotateMotor(MAX_MOTOR_SPEED, -MAX_MOTOR_SPEED);
    }
    else if (results.value == 0x40BD41B)
    {
      rotateMotor(-MAX_MOTOR_SPEED, MAX_MOTOR_SPEED);
    }
    delay(200);
  }
  else
  {
    rotateMotor(0, 0);
  }
}
```

• Line Following Robot

• Components :

1. ARDUINO UNO
2. L298D MOTOR DRIVER
3. 2X IR PROXIMITY SENSOR
4. DC MOTOR

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
#define IR_SENSOR_RIGHT 11
#define IR_SENSOR_LEFT 12
#define MOTOR_SPEED 180

//Right motor
int enableRightMotor=6;
int rightMotorPin1=7;
int rightMotorPin2=8;

//Left motor
int enableLeftMotor=5;
int leftMotorPin1=9;
int leftMotorPin2=10;

void setup()
{
    //The problem with TT gear motors is that, at very low pwm value it does not even rotate.
    //If we increase the PWM value then it rotates faster and our robot is not controlled in that speed and goes out of line.
    //For that we need to increase the frequency of analogWrite.
    //Below line is important to change the frequency of PWM signal on pin D5 and D6
    //Because of this, motor runs in controlled manner (lower speed) at high PWM value.
    //This sets frequency as 7812.5 hz.
    TCCR0B = TCCR0B & B11111000 | B00000010 ;

    // put your setup code here, to run once:
    pinMode(enableRightMotor, OUTPUT);
    pinMode(rightMotorPin1, OUTPUT);
    pinMode(rightMotorPin2, OUTPUT);

    pinMode(enableLeftMotor, OUTPUT);
    pinMode(leftMotorPin1, OUTPUT);
    pinMode(leftMotorPin2, OUTPUT);

    pinMode(IR_SENSOR_RIGHT, INPUT);
    pinMode(IR_SENSOR_LEFT, INPUT);
    rotateMotor(0,0);
}

void loop()
{
    int rightIRSensorValue = digitalRead(IR_SENSOR_RIGHT);
    int leftIRSensorValue = digitalRead(IR_SENSOR_LEFT);

    //If none of the sensors detects black line, then go straight
    if (rightIRSensorValue == LOW && leftIRSensorValue == LOW)
    {
        rotateMotor(MOTOR_SPEED, MOTOR_SPEED);
    }
    //If right sensor detects black line, then turn right
    else if (rightIRSensorValue == HIGH && leftIRSensorValue == LOW )
    {
        rotateMotor(-MOTOR_SPEED, MOTOR_SPEED);
    }
    //If left sensor detects black line, then turn left
    else if (rightIRSensorValue == LOW && leftIRSensorValue == HIGH )
    {
        rotateMotor(MOTOR_SPEED, -MOTOR_SPEED);
    }
    //If both the sensors detect black line, then stop
    else
    {
        rotateMotor(0, 0);
    }
}

void rotateMotor(int rightMotorSpeed, int leftMotorSpeed)
{
    if (rightMotorSpeed < 0)
    {
        digitalWrite(rightMotorPin1,LOW);
        digitalWrite(rightMotorPin2,HIGH);
    }
    else if (rightMotorSpeed > 0)
    {
        digitalWrite(rightMotorPin1,HIGH);
        digitalWrite(rightMotorPin2,LOW);
    }
    else
    {
        digitalWrite(rightMotorPin1,LOW);
        digitalWrite(rightMotorPin2,LOW);
    }

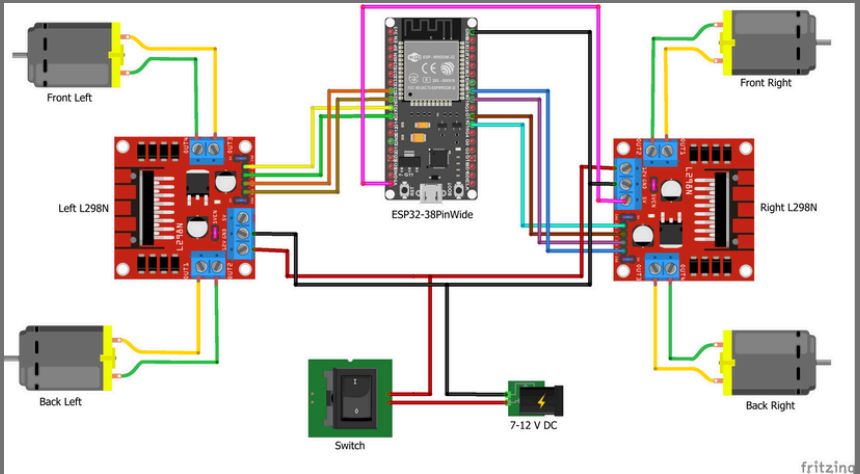
    if (leftMotorSpeed < 0)
    {
        digitalWrite(leftMotorPin1,LOW);
        digitalWrite(leftMotorPin2,HIGH);
    }
    else if (leftMotorSpeed > 0)
    {
        digitalWrite(leftMotorPin1,HIGH);
        digitalWrite(leftMotorPin2,LOW);
    }
    else
    {
        digitalWrite(leftMotorPin1,LOW);
        digitalWrite(leftMotorPin2,LOW);
    }
    analogWrite(enableRightMotor, abs(rightMotorSpeed));
    analogWrite(enableLeftMotor, abs(leftMotorSpeed));
}
```

• WiFi Car

• Components :

1. ESP32 WI-FI MODULE
2. 2x L298D MOTOR DRIVER
3. DC MOTOR

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
#include <Arduino.h>
#include ESP8266
#include <WiFi.h>
#include <AsyncTCP.h>
//if defined(ESP8266)
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#endif
#include <ESPAsyncWebServer.h>

#define UP 1
#define DOWN 2
#define LEFT 3
#define RIGHT 4
#define UP_LEFT 5
#define UP_RIGHT 6
#define DOWN_LEFT 7
#define DOWN_RIGHT 8
#define TURN_LEFT 9
#define TURN_RIGHT 10
#define STOP 0

#define FRONT_RIGHT_MOTOR 0
#define BACK_RIGHT_MOTOR 1
#define FRONT_LEFT_MOTOR 2
#define BACK_LEFT_MOTOR 3

#define FORWARD 1
#define BACKWARD -1

struct MOTOR_PINS
{
    int pinIN1;
    int pinIN2;
};

std::vector<MOTOR_PINS> motorPins =
{
    {16, 17}, //FRONT_RIGHT_MOTOR
    {19, 18}, //BACK_RIGHT_MOTOR
    {27, 26}, //FRONT_LEFT_MOTOR
    {25, 33}, //BACK_LEFT_MOTOR
};

const char* ssid = "MyWiFiCar";
const char* password = "12345678";

AsyncWebServer server(80);
AsyncWebsocket ws("/ws");

const char* htmlHomePage PROGMEM = R"HTMLHOMEPAGE(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no">
<style>
.arrows {
font-size:70px;
color:red;
}
.circularArrows {
font-size:80px;
color:blue;
}
td {
background-color:black;
border-radius:25%;
box-shadow: 5px 5px #888888;
}
td:active {
transform: translate(5px,5px);
box-shadow: none;
}
-noselect {
-webkit-touch-callout: none; /* iOS Safari */
-webkit-user-select: none; /* Safari */
-khtml-user-select: none; /* Konqueror HTML */
-moz-user-select: none; /* Firefox */
-ms-user-select: none; /* Internet Explorer/Edge */
user-select: none; /* Non-prefixed version, currently
supported by Chrome and Opera */
}
</style>
</head>
<body class="noselect" align="center" style="background-color:white">

<h1 style="color: teal;text-align:center;">Hash Include Electronics</h1>
<h2 style="color: teal;text-align:center;">Wi-Fi &#128663; Control</h2>

<table id="mainTable" style="width:480px;margin:auto;table-layout:fixed" CELLSPACING=10>
<tr>
<td ontouchstart="onTouchStart&End('5')" ontouchend="onTouchStart&End('0')"><span class="arrows"> &#110177;</span></td>
<td ontouchstart="onTouchStart&End('1')" ontouchend="onTouchStart&End('0')"><span class="arrows"> &#10879;</span></td>
<td ontouchstart="onTouchStart&End('6') ontouchend="onTouchStart&End('0')"><span class="arrows"> &#11016;</span></td>
</tr>

<tr>
<td ontouchstart="onTouchStart&End('3') ontouchend="onTouchStart&End('0')"><span class="arrows"> &#10678;</span></td>
<td></td>
<td ontouchstart="onTouchStart&End('4') ontouchend="onTouchStart&End('0')"><span class="arrows"> &#10680;</span></td>
</tr>

<tr>
<td ontouchstart="onTouchStart&End('7') ontouchend="onTouchStart&End('0')"><span class="arrows"> &#11019;</span></td>
<td ontouchstart="onTouchStart&End('2') ontouchend="onTouchStart&End('0')"><span class="arrows"> &#10881;</span></td>
<td ontouchstart="onTouchStart&End('8') ontouchend="onTouchStart&End('0')"><span class="arrows"> &#11018;</span></td>
</tr>

<tr>
<td ontouchstart="onTouchStart&End('9') ontouchend="onTouchStart&End('0')"><span class="circularArrows"> &#10634;</span></td>
<td style="background-color:white;box-shadow:none"></td>
<td ontouchstart="onTouchStart&End('10') ontouchend="onTouchStart&End('0')"><span class="circularArrows"> &#10635;</span></td>
</tr>
</table>

<script>
var websocketUrl = "ws://" + window.location.hostname + "/ws";
var websocket;

function initWebSocket()
{
    websocket = new WebSocket(websocketUrl);
    websocket.onopen = function(event){};
    websocket.onclose = function(event){setTimeout(initWebSocket, 2000)};
    websocket.onmessage = function(event){};
}

function onTouchStart&End(value)
{
    websocket.send(value);
}

window.onload = initWebSocket;
document.getElementById("mainTable").addEventListener('touchend', function(event){
    event.preventDefault();
});
</script>

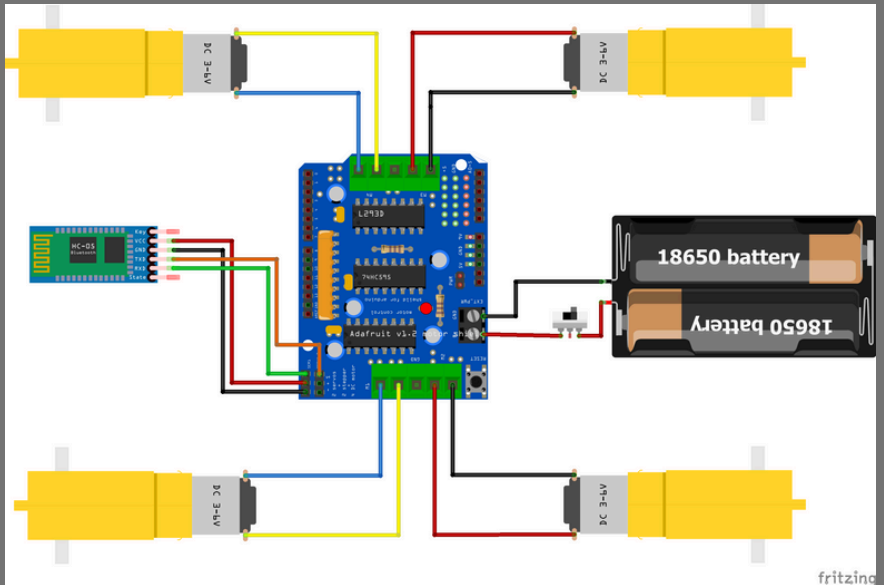
</body>
</html>
)HTMLHOMEPAGE;
```


- **Bluetooth control car**

- **Components :**

1. ARDUNIO UNO
2. HC-05 BLUETOOTH MODULE
3. L293D MOTOR DRIVER
4. 4X DC MOTOR

- **Circuit Diagram :**



- **Main Code :**

goto next page !

• Main Code :

```
#include <AFMotor.h>
#include <SoftwareSerial.h>

SoftwareSerial bluetoothSerial(9, 10); // RX, TX

//initial motors pin
AF_DCMotor motor1(1, MOTOR12_1KHZ);
AF_DCMotor motor2(2, MOTOR12_1KHZ);
AF_DCMotor motor3(3, MOTOR34_1KHZ);
AF_DCMotor motor4(4, MOTOR34_1KHZ);

char command;

void setup()
{
    bluetoothSerial.begin(9600); //Set the baud rate to your Bluetooth module.
}

void loop() {
    if (bluetoothSerial.available() > 0) {
        command = bluetoothSerial.read();

        Stop(); //initialize with motors stoped

        switch (command) {
            case 'F':
                forward();
                break;
            case 'B':
                back();
                break;
            case 'L':
                left();
                break;
            case 'R':
                right();
                break;
        }
    }
}

void forward()
{
    motor1.setSpeed(255); //Define maximum velocity
    motor1.run(FORWARD); //rotate the motor clockwise
    motor2.setSpeed(255); //Define maximum velocity
    motor2.run(FORWARD); //rotate the motor clockwise
    motor3.setSpeed(255); //Define maximum velocity
    motor3.run(FORWARD); //rotate the motor clockwise
    motor4.setSpeed(255); //Define maximum velocity
    motor4.run(FORWARD); //rotate the motor clockwise
}

void back()
{
    motor1.setSpeed(255); //Define maximum velocity
    motor1.run(BACKWARD); //rotate the motor anti-clockwise
    motor2.setSpeed(255); //Define maximum velocity
    motor2.run(BACKWARD); //rotate the motor anti-clockwise
    motor3.setSpeed(255); //Define maximum velocity
    motor3.run(BACKWARD); //rotate the motor anti-clockwise
    motor4.setSpeed(255); //Define maximum velocity
    motor4.run(BACKWARD); //rotate the motor anti-clockwise
}

void left()
{
    motor1.setSpeed(255); //Define maximum velocity
    motor1.run(BACKWARD); //rotate the motor anti-clockwise
    motor2.setSpeed(255); //Define maximum velocity
    motor2.run(BACKWARD); //rotate the motor anti-clockwise
    motor3.setSpeed(255); //Define maximum velocity
    motor3.run(FORWARD); //rotate the motor clockwise
    motor4.setSpeed(255); //Define maximum velocity
    motor4.run(FORWARD); //rotate the motor clockwise
}

void right()
{
    motor1.setSpeed(255); //Define maximum velocity
    motor1.run(FORWARD); //rotate the motor clockwise
    motor2.setSpeed(255); //Define maximum velocity
    motor2.run(FORWARD); //rotate the motor clockwise
    motor3.setSpeed(255); //Define maximum velocity
    motor3.run(BACKWARD); //rotate the motor anti-clockwise
    motor4.setSpeed(255); //Define maximum velocity
    motor4.run(BACKWARD); //rotate the motor anti-clockwise
}

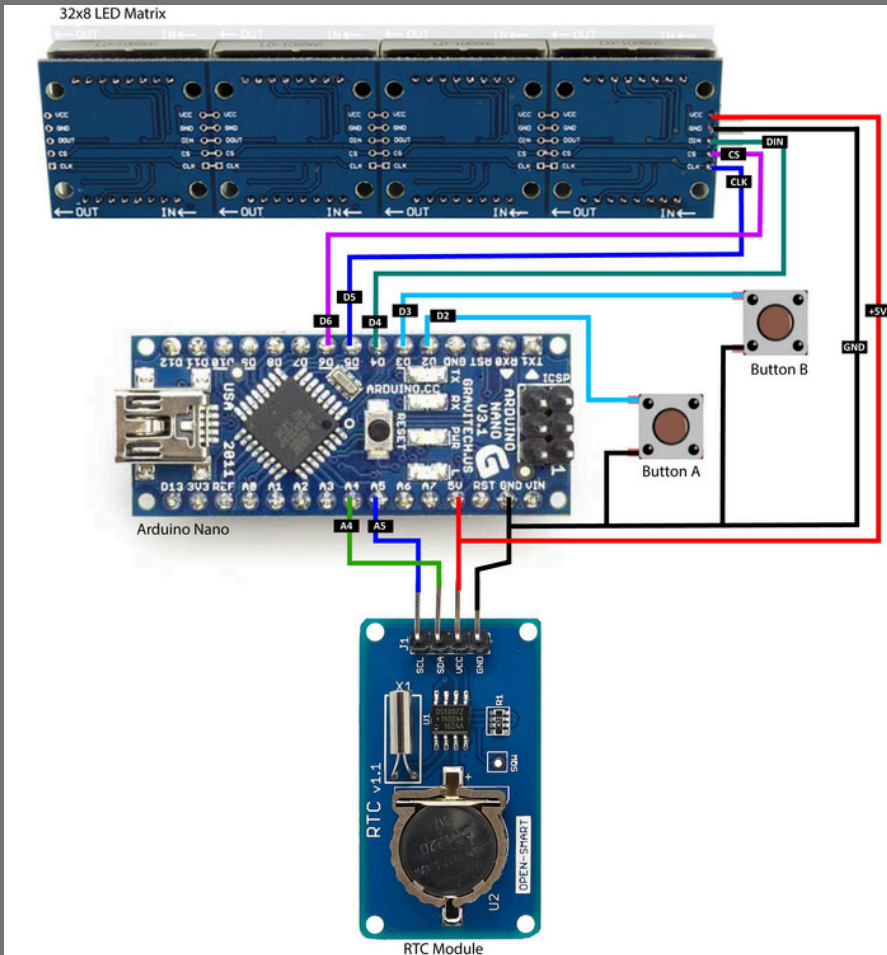
void Stop()
{
    motor1.setSpeed(0); //Define minimum velocity
    motor1.run(RELEASE); //stop the motor when release the button
    motor2.setSpeed(0); //Define minimum velocity
    motor2.run(RELEASE); //rotate the motor clockwise
    motor3.setSpeed(0); //Define minimum velocity
    motor3.run(RELEASE); //stop the motor when release the button
    motor4.setSpeed(0); //Define minimum velocity
    motor4.run(RELEASE); //stop the motor when release the button
}
```

- **Digital clock**

- **Components :**

1. ARDUNIO NANO
2. RTC MODULE
3. 32*2 MATRIX DISPLAY
4. PUSH BUTTON

- **Circuit Diagram :**



- **Main Code :**

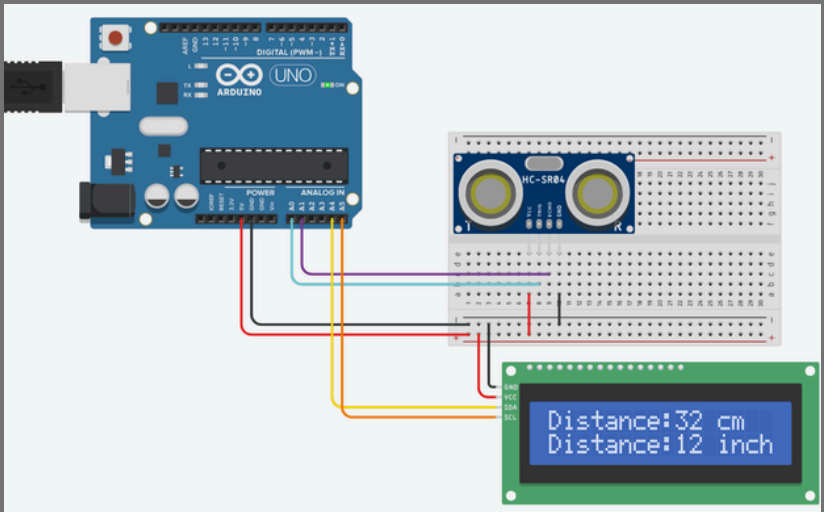
DOWNLOAD LINK - [Main Code font.h](#)

- Distance Measurement using Ultrasonic sensor

- **Components :**

1. ARDUNIO UNO
2. ULTRASONIC SENSOR
3. 16X2 DISPLAY
4. BREADBOARD

- **Circuit Diagram :**



- **Main Code :**

goto next page !

• Main Code :

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

#define trigPin A0 //Sensor Trig pin connected to Arduino pin A0
#define echoPin A1 //Sensor Echo pin connected to Arduino pin A1

long distanceInch;

void setup()
{
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    lcd.init();
    lcd.backlight();
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Simple  Circuits");
    delay(2000);
    lcd.clear();
    lcd.setCursor(0,0); //Set LCD cursor to upper left corner, column 0, row 0
    lcd.print("Distance:");//Print Message on First Row
    lcd.setCursor(0,1);
    lcd.print("Distance:");
}

void loop()
{
    long duration, distance;
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);
    distance = (duration/2) / 29.1;
    distanceInch = duration*0.0133/2;

    lcd.setCursor(9,0);
    lcd.print("                ");
    lcd.setCursor(9,0);
    lcd.print(                distance); //Print measured distance
    lcd.print(" cm"); //Print your units.
    lcd.setCursor(9,1); //
    lcd.print("                "); //Print blanks to clear the row
    lcd.setCursor(9,1);
    lcd.print(distanceInch);
    lcd.print(" inch"); //Print your units.

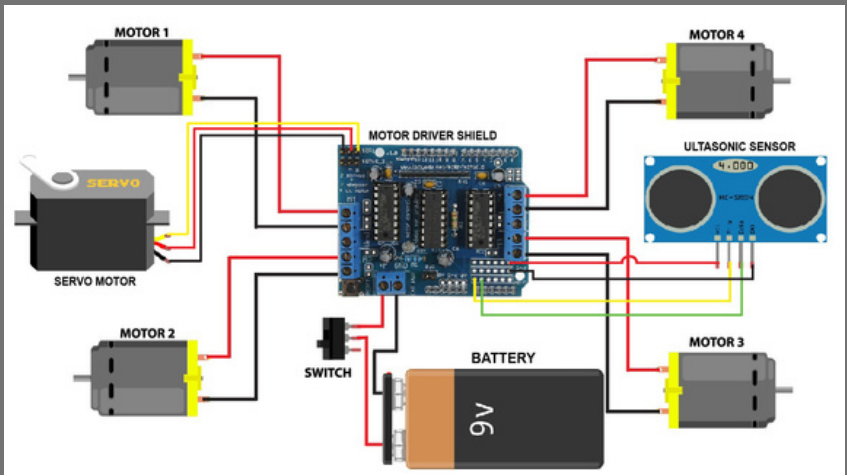
    delay(200); //pause to let things settle
}
```

• Object Avoiding Robot

• Components :

1. ARDUNIO UNO
2. ULTRASONIC SENSOR
3. L293D MOTOR DRIVER
4. SERVO MOTOR
5. 4x DC MOTOR

• Circuit Diagram :



• Main Code :

goto next page !

• Main Code :

```
// Before uploading the code you have to install the necessary library
//Robot Library https://forum.arduino.cc/default-motor-shield-library-install
//NewPing Library https://github.com/livetronic/Arduino-NewPing
//Servo Library https://github.com/arduino-libraries/Servo.git

#include <Motor.h>
#include <NewPing.h>
#include <Servo.h>

#define TRIG_PIN A0
#define ECHO_PIN A1
#define MAX_DISTANCE 200
#define MAX_SPEED 100 // sets speed of DC motors
#define MAX_SPEED_OFFSET 20

NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

//_D_Motor motor(1, MOTOR12_16K2);
//_D_Motor motor(2, MOTOR12_16K2);
//_D_Motor motor(3, MOTOR14_16K2);
//_D_Motor motor(4, MOTOR14_16K2);
Servo myservo;

boolean goeForward=false;
int distance = 100;
int speedSet = 0;

void setup() {
  myservo.attach(10);
  myservo.write(15);
  delay(2000);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
}

void loop() {
  int distance = 0;
  int distancec = 0;
  delay(40);

  if(distance<=15)
  {
    moveStop();
    delay(100);
    moveBackward();
    delay(500);
    moveStop();
    delay(500);
    distancec = lookRight();
    delay(200);
    distancec = lookLeft();
    delay(200);

    if(distance<=distancec)
    {
      turnRight();
      moveUp();
    }
    else
    {
      turnLeft();
      moveUp();
    }
  }
  else
  {
    moveForward();
  }
  distance = readPing();
}

int lookRight()
{
  myservo.write(90);
  delay(500);
  int distance = readPing();
  delay(100);
  myservo.write(115);
  return distance;
}

int lookLeft()
{
  myservo.write(170);
  delay(500);
  int distance = readPing();
  delay(100);
  myservo.write(115);
  return distance;
  delay(100);
}

int readPing() {
  delay(70);
  int cm = sonar.ping_cm();
  if(cm==0)
  {
    cm = 255;
  }
  return cm;
}

void moveUp() {
  motor1.run(RELEASE);
  motor2.run(RELEASE);
  motor3.run(RELEASE);
  motor4.run(RELEASE);
}

void moveForward() {
  if(goeForward)
  {
    goeForward=true;
    motor1.run(FORWARD);
    motor2.run(FORWARD);
    motor3.run(FORWARD);
    motor4.run(FORWARD);
    for (speedSet = 0; speedSet < MAX_SPEED; speedSet +=2) // slowly bring the speed up to avoid loading down the batteries too quickly
    {
      motor1.setSpeed(speedSet);
      motor2.setSpeed(speedSet);
      motor3.setSpeed(speedSet);
      motor4.setSpeed(speedSet);
      delay(5);
    }
  }
}

void moveBackward() {
  goeForward=false;
  motor1.run(BACKWARD);
  motor2.run(BACKWARD);
  motor3.run(BACKWARD);
  motor4.run(BACKWARD);
  for (speedSet = 0; speedSet < MAX_SPEED; speedSet +=2) // slowly bring the speed up to avoid loading down the batteries too quickly
  {
    motor1.setSpeed(speedSet);
    motor2.setSpeed(speedSet);
    motor3.setSpeed(speedSet);
    motor4.setSpeed(speedSet);
    delay(5);
  }
}

void turnRight() {
  motor1.run(FORWARD);
  motor2.run(FORWARD);
  motor3.run(BACKWARD);
  motor4.run(BACKWARD);
  delay(500);
  motor1.run(FORWARD);
  motor2.run(FORWARD);
  motor3.run(FORWARD);
  motor4.run(FORWARD);
}

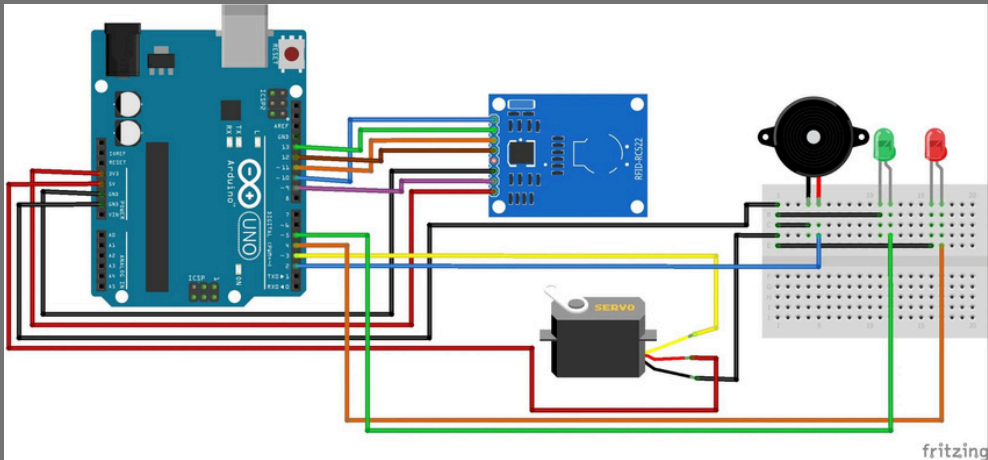
void turnLeft() {
  motor1.run(BACKWARD);
  motor2.run(BACKWARD);
  motor3.run(FORWARD);
  motor4.run(FORWARD);
  delay(500);
  motor1.run(FORWARD);
  motor2.run(FORWARD);
  motor3.run(FORWARD);
  motor4.run(FORWARD);
}
```

• RFID Door Lock System

• Components :

1. ARDUNIO UNO
2. RFID TAG
3. SERVO MOTOR
4. BUZZER
5. 5MM LEDS
6. BREADBOARD

• Circuit Diagram :



• Main Code :

goto next page !

```

#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>

#define SS_PIN 10
#define RST_PIN 9
#define LED_G 4 //define green LED pin
#define LED_R 5 //define red LED
#define BUZZER 2 //buzzer pin
MFRC522 mfrc522(SS_PIN, RST_PIN); // Create MFRC522 instance.
Servo myServo; //define servo name

void setup()
{
  Serial.begin(9600); // Initiate a serial communication
  SPI.begin(); // Initiate SPI bus
  mfrc522.PCD_Init(); // Initiate MFRC522
  myServo.attach(3); //servo pin
  myServo.write(0); //servo start position
  pinMode(LED_G, OUTPUT);
  pinMode(LED_R, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  noTone(BUZZER);
  Serial.println("Put your card to the reader...");
  Serial.println();
}

void loop()
{
  // Look for new cards
  if ( ! mfrc522.PICC_IsNewCardPresent())
  {
    return;
  }
  // Select one of the cards
  if ( ! mfrc522.PICC_ReadCardSerial())
  {
    return;
  }
  //Show UID on serial monitor
  Serial.print("UID tag :");
  String content= "";
  byte letter;
  for (byte i = 0; i < mfrc522.uid.size; i++)
  {
    Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " ");
    Serial.print(mfrc522.uid.uidByte[i], HEX);
    content.concat(String(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " "));
    content.concat(String(mfrc522.uid.uidByte[i], HEX));
  }
  Serial.println();
  Serial.print("Message : ");
  content.toUpperCase();
  if (content.substring(1) == "83 23 38 BB") //change here the UID of the card/cards that you want to give access
  {
    Serial.println("Authorized access");
    Serial.println();
    delay(500);
    digitalWrite(LED_G, HIGH);
    tone(BUZZER, 500);
    delay(300);
    noTone(BUZZER);
    myServo.write(180);
    delay(5000);
    myServo.write(0);
    digitalWrite(LED_G, LOW);
  }
  else
  {
    Serial.println(" Access denied");
    digitalWrite(LED_R, HIGH);
    tone(BUZZER, 300);
    delay(1000);
    digitalWrite(LED_R, LOW);
    noTone(BUZZER);
  }
}

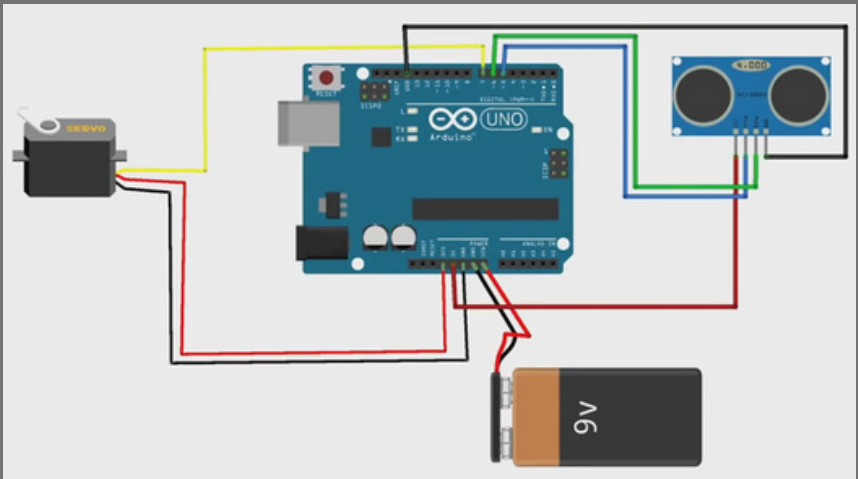
```

- **Smart Dustbin**

- **Components :**

1. ARDUNIO UNO
2. ULTRASONIC SENSOR
3. SERVO MOTOR

- **Circuit Diagram :**



- **Main Code :**

goto next page !

```

#include <Servo.h>    //servo library
Servo servo;
int trigPin = 5;
int echoPin = 6;
int servoPin = 7;
int led= 10;
long duration, dist, average;
long aver[3];    //array for average

void setup() {
    Serial.begin(9600);
    servo.attach(servoPin);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    servo.write(0);          //close cap on power on
    delay(100);
    servo.detach();
}

void measure() {
    digitalWrite(10,HIGH);
    digitalWrite(trigPin, LOW);
    delayMicroseconds(5);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(15);
    digitalWrite(trigPin, LOW);
    pinMode(echoPin, INPUT);
    duration = pulseIn(echoPin, HIGH);
    dist = (duration/2) / 29.1;    //obtain distance
}

void loop() {
    for (int i=0;i<=2;i++) {    //average distance
        measure();
        aver[i]=dist;
        delay(10);            //delay between measurements
    }
    dist=(aver[0]+aver[1]+aver[2])/3;

    if ( dist<50 ) {
        //Change distance as per your need
        servo.attach(servoPin);
        delay(1);
        servo.write(0);
        delay(3000);
        servo.write(150);
        delay(1000);
        servo.detach();
    }
    Serial.print(dist);
}

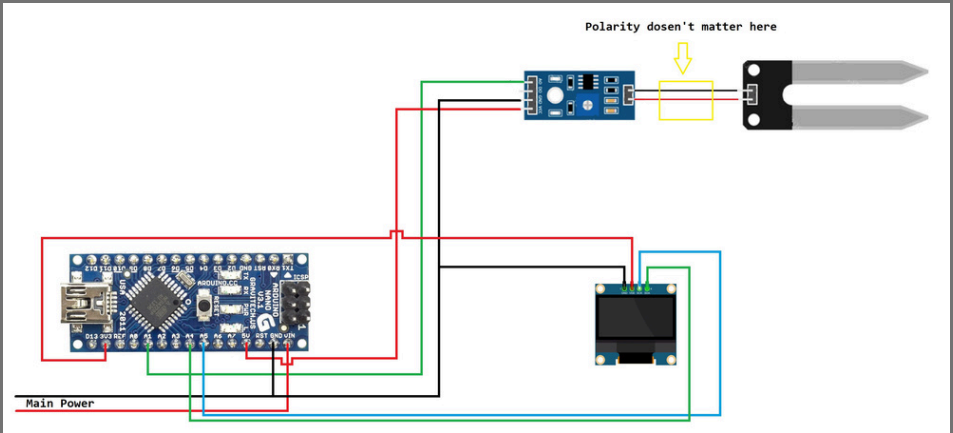
```

• Soil Moisture Sensor

• Components :

1. ARDUNIO NANO
2. 0.91 INCH OLED DISPLAY
3. SOIL MOISTURE SENSOR

• Circuit Diagram :



• Main Code :

goto next page !

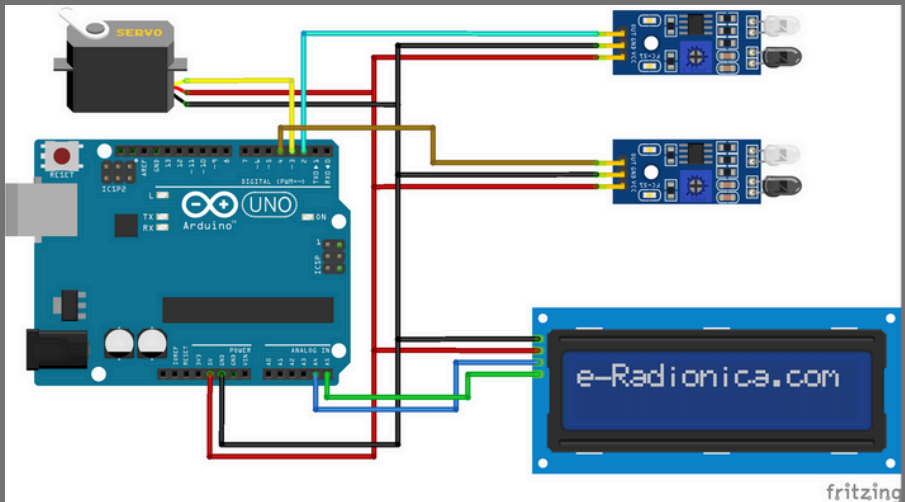
```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#define OLED_RESET 4
#define sensor A1
Adafruit_SSD1306 display(OLED_RESET);
void setup()
{
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C); //address(0x3C)
    display.clearDisplay();
}
void loop()
{
    int value = analogRead(sensor);
    int percent = map(value, 1024, 0, 0, 100);
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(0,0);
    display.println("MOISTURE");
    display.print(percent);
    display.print("%");
    display.display();
    display.clearDisplay();
}
```

• Arduino Car Parking System

• Components :

1. ARDUINO UNO
2. IR PROXIMITY SENSOR
3. SERVO MOTOR
4. 16x2 LCD I2C DISPLAY
5. JUMPERS

• Circuit Diagram :



• Main Code :

goto next page !

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x3F,16,2); //Change the HEX address
#include <Servo.h>

Servo myservo1;

int IR1 = 2;
int IR2 = 4;

int Slot = 4;           //Enter Total number of parking Slots

int flag1 = 0;
int flag2 = 0;

void setup() {
  lcd.begin();
  lcd.backlight();
  pinMode(IR1, INPUT);
  pinMode(IR2, INPUT);

  myservo1.attach(3);
  myservo1.write(100);

  lcd.setCursor (0,0);
  lcd.print("    ARDUINO    ");
  lcd.setCursor (0,1);
  lcd.print(" PARKING SYSTEM ");
  delay (2000);
  lcd.clear();
}

void loop(){

  if(digitalRead (IR1) == LOW && flag1==0){
    if(Slot>0){flag1=1;
    if(flag2==0){myservo1.write(0); Slot = Slot-1;}
    }else{
    lcd.setCursor (0,0);
    lcd.print("    SORRY :(    ");
    lcd.setCursor (0,1);
    lcd.print(" Parking Full ");
    delay (3000);
    lcd.clear();
    }
  }

  if(digitalRead (IR2) == LOW && flag2==0){flag2=1;
  if(flag1==0){myservo1.write(0); Slot = Slot+1;}
  }

  if(flag1==1 && flag2==1){
    delay (1000);
    myservo1.write(100);
    flag1=0, flag2=0;
  }

  lcd.setCursor (0,0);
  lcd.print("    WELCOME!    ");
  lcd.setCursor (0,1);
  lcd.print("Slot Left: ");
  lcd.print(Slot);
}

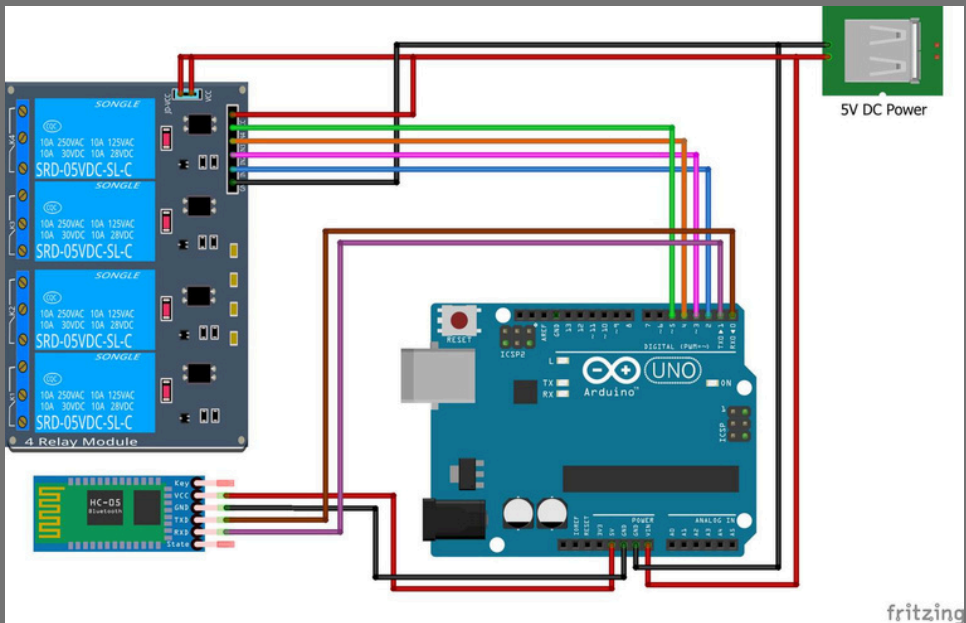
```

• Arduino Voice Control Home Automation

• Components :

1. ARDUINO UNO
2. HC-05 BLUETOOTH MODULE
3. 4-CH RELAY MODULE
4. BREADBOARD

• Circuit Diagram :



• Main Code :

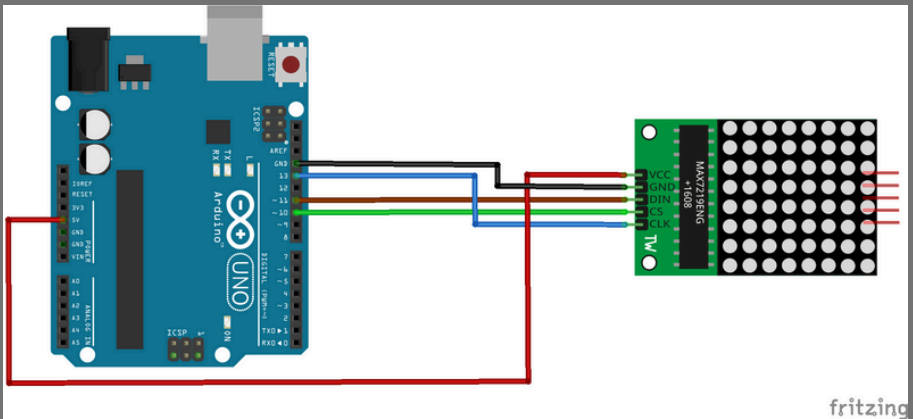
goto next page !

- **Arduino Text Scrolling Message Display**

- **Components :**

1. ARDUINO
2. DOT MATRIX 4 CHANNEL DISPLAY

- **Circuit Diagram :**



- **Main Code :**

goto next page !

```

#include <Adafruit_GFX.h>
#include <Adafruit_NeoMatrix.h>
#include <Adafruit_NeoPixel.h>
#ifdef PSTR
  #define PSTR
#endif

#define PIN 8
#define mw 10
#define mh 5

Adafruit_NeoMatrix matrix = Adafruit_NeoMatrix(mw, mh, PIN,
NEO_MATRIX_ZIGZAG,
  NEO_GRB          + NEO_KHZ400);

const uint16_t colors[] = {
  matrix.Color(255, 0, 0), matrix.Color(0, 255, 0), matrix.Color(0, 0, 255)
};

void setup() {
  matrix.begin();
  matrix.setTextWrap(false);
  matrix.setBrightness(40);
  matrix.setTextColor(colors[0]);
}

int x    = matrix.width();
int pass = 0;

void loop() {
  matrix.fillScreen(0);
  matrix.setCursor(x,0);
  matrix.print(F("enter your text"));
  if(--x < -50) {
    x = matrix.width();
    if(++pass >= 3) pass = 0;
    matrix.setTextColor(colors[pass]);
  }
  matrix.show();
  delay(70);
}

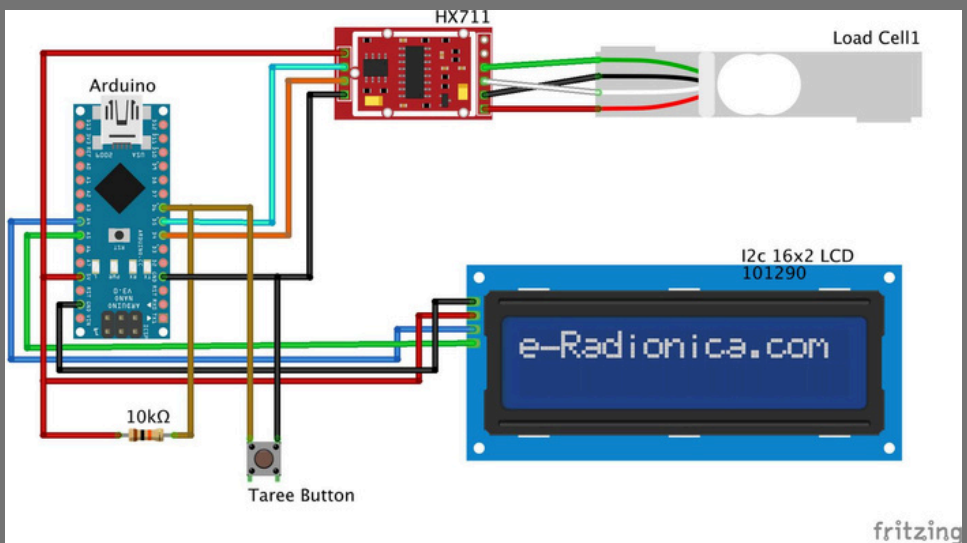
```

• Arduino Digital Weight Scale

• Components :

1. ARDUINO NANO
2. 16x2 I2C LCD DISPLAY
3. HX711 MODULE
4. 5KG LOAD CELL
5. BREADBOARD

• Circuit Diagram :



• Main Code :

goto next page !

```

//Arduino Digital Weight Scale HX711 Load Cell Module
#include <HX711_ADC.h> // https://github.com/olka1/HX711_ADC
#include <Wire.h>
#include <LiquidCrystal_I2C.h> // LiquidCrystal_I2C library
HX711_ADC LoadCell(4, 5); // dt pin, sck pin
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD HEX address 0x27
int taree = 6;
int a = 0;
float b = 0;

void setup() {
  pinMode (taree, INPUT_PULLUP);
  LoadCell.begin(); // start connection to HX711
  LoadCell.start(1000); // load cells gets 1000ms of time to stabilize

  ///////////////////////////////////////////////////
  LoadCell.setCalFactor(375); // Calibarate your LOAD CELL with 100g weight,
  and change the value according to readings
  ///////////////////////////////////////////////////

  lcd.begin(); // begins connection to the LCD module
  lcd.backlight(); // turns on the backlight
  lcd.setCursor(1, 0); // set cursor to first row
  lcd.print("Digital Scale "); // print out to LCD
  lcd.setCursor(0, 1); // set cursor to first row
  lcd.print(" 5KG MAX LOAD "); // print out to LCD
  delay(3000);
  lcd.clear();
}

void loop() {
  lcd.setCursor(1, 0); // set cursor to first row
  lcd.print("Digital Scale "); // print out to LCD
  LoadCell.update(); // retrieves data from the load cell
  float i = LoadCell.getData(); // get output value
  if (i<0)
  {
    i = i * (-1);
    lcd.setCursor(0, 1);
    lcd.print("-");
    lcd.setCursor(8, 1);
    lcd.print("-");
  }
  else
  {
    lcd.setCursor(0, 1);
    lcd.print(" ");
    lcd.setCursor(8, 1);
    lcd.print(" ");
  }

  lcd.setCursor(1, 1); // set cursor to secon row
  lcd.print(i, 1); // print out the retrieved value to the second row
  lcd.print("g ");
  float z = i/28.3495;
  lcd.setCursor(9, 1);
  lcd.print(z, 2);
  lcd.print("oz ");

  if (i>=5000)
  {
    i=0;
    lcd.setCursor(0, 0); // set cursor to secon row
    lcd.print(" Over Loaded ");
    delay(200);
  }
  if (digitalRead (taree) == LOW)
  {
    lcd.setCursor(0, 1); // set cursor to secon row
    lcd.print(" Taring... ");
    LoadCell.start(1000);
    lcd.setCursor(0, 1);
    lcd.print(" ");
  }
}

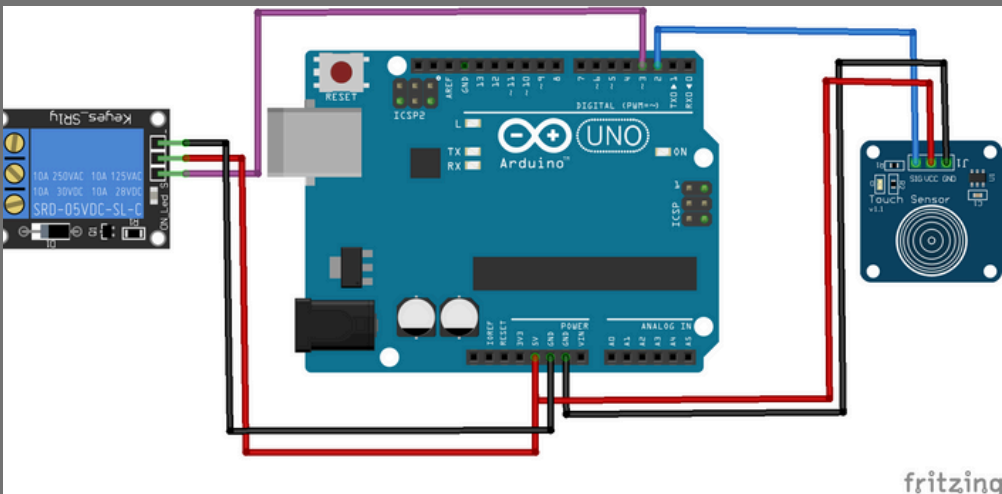
```

• Arduino Touch Sensor

• Components :

1. ARDUINO
2. TOUCH SENSOR TTP223B
3. RELAY
4. LED
5. BREADBOARD
6. JUMPERS

• Circuit Diagram :



• Main Code :

goto next page !

```
int touchPin = 2;
int relayPin = 3;

int val = 0;
int lightON = 0;
int touched = 0;

void setup() {
  Serial.begin(9600);
  pinMode(touchPin, INPUT);
  pinMode(relayPin, OUTPUT);
}

void loop() {

  val = digitalRead(touchPin);

  if(val == HIGH && lightON == LOW){

    touched = 1-touched;
    delay(100);
  }

  lightON = val;

  if(touched == HIGH){
    Serial.println("Light ON");
    digitalWrite(relayPin, LOW);

  }else{
    Serial.println("Light OFF");
    digitalWrite(relayPin, HIGH);

  }

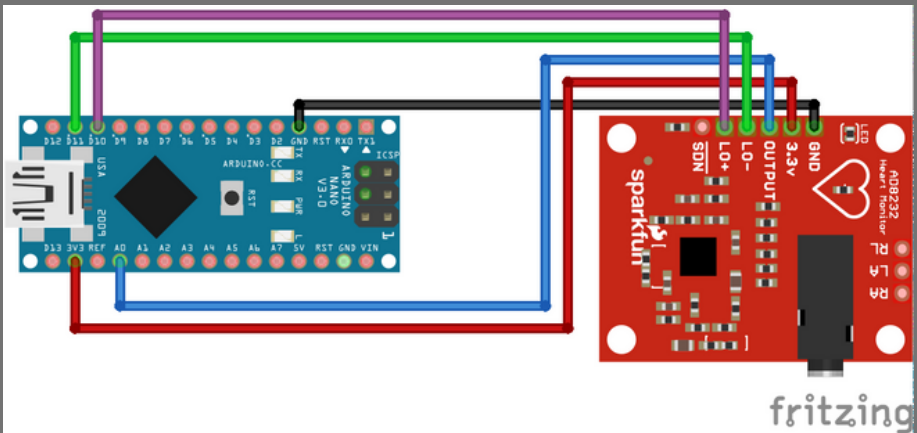
  delay(100);
}
```

• Arduino ECG Heart Rate Monitor AD8232

• Components :

1. ARDUINO NANO
2. ECG AD8232 SENSOR
3. BREADBOARD
4. WIRES

• Circuit Diagram :



• Main Code :

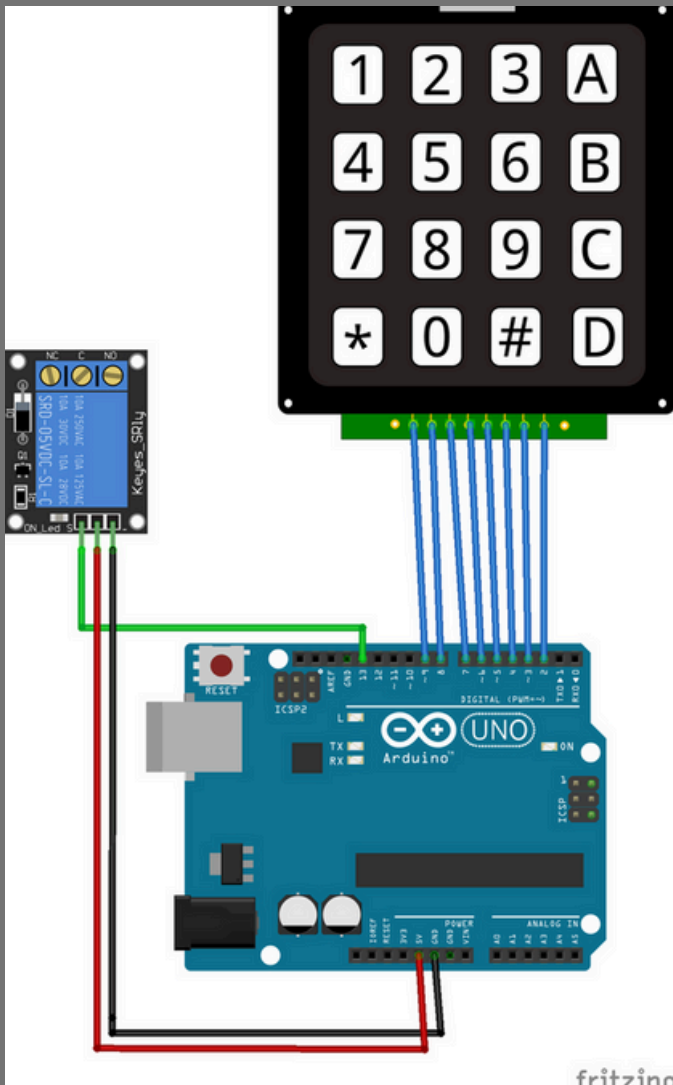
```
void setup() {  
  // initialize the serial communication:  
  Serial.begin(9600);  
  pinMode(10, INPUT); // Setup for leads off detection LO +  
  pinMode(11, INPUT); // Setup for leads off detection LO -  
}  
  
void loop() {  
  
  if((digitalRead(10) == 1)|| (digitalRead(11) == 1)){  
    Serial.println('!');  
  }  
  else{  
    // send the value of analog input 0:  
    Serial.println(analogRead(A0));  
  }  
  //Wait for a bit to keep serial data from saturating  
  delay(1);  
}
```

• Arduino Keypad Solenoid Lock

• Components :

1. ARDUINO
2. SOLENOID LOCK
3. 4X4 KEYPAD
4. RELAY MODULE
5. JUMPERS

• Circuit Diagram :



goto next page !

- **Main Code :**

```
#include <Keypad.h>
char* password = "123"; // change the password here, just pick any 3
numbers
int position = 0;
const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
{'1','2','3','A'},
{'4','5','6','B'},
{'7','8','9','C'},
{'*','0','#','D'}
};

byte rowPins[ROWS] = { 9, 8, 7, 6 };
byte colPins[COLS] = { 5, 4, 3, 2 };
Keypad keypad = Keypad( makeKeymap(keys), rowPins, colPins, ROWS, COLS );
int Lock = 13;

void setup()
{
pinMode(Lock, OUTPUT);
LockedPosition(true);
}

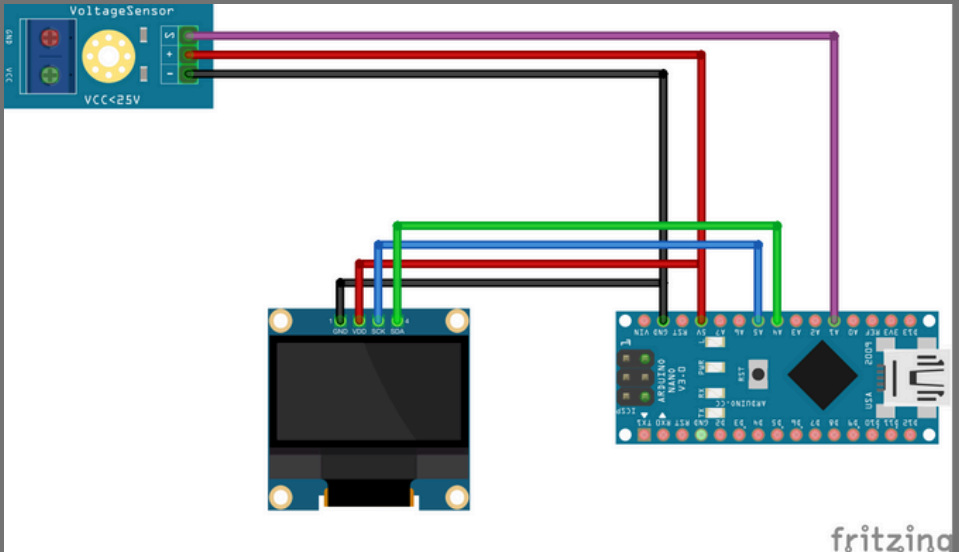
void loop()
{
char key = keypad.getKey();
if (key == '*' || key == '#')
{
position = 0;
LockedPosition(true);
}
if (key == password[position])
{
position ++;
}
if (position == 3)
{
LockedPosition(false);
}
delay(100);
}
void LockedPosition(int locked)
{
if (locked)
{
digitalWrite(Lock, LOW);
}
else
{
digitalWrite(Lock, HIGH);
}
}
```

• Arduino Voltmeter OLED Display

• Components :

1. ARDUINO NANO
2. OLED DISPLAY
3. VOLTAGE SENSOR
4. BREADBOARD
5. WIRES

• Circuit Diagram :



• Main Code :

goto next page !

```

#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_RESET          A4

#define SSD1306_WHITE 1
#define SSD1306_BLACK 0
#define SSD1306_INVERSE 2

Adafruit_SSD1306 display(OLED_RESET);

int analogInput = A1; //Sensor Input

void setup() {
  pinMode(analogInput, INPUT);

  Serial.begin(9600);
  Serial.println("VOLTMETER");

  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

  display.display();
  display.clearDisplay();

  display.display();

  display.drawRect(0, 0, 128, 12, SSD1306_WHITE);
  display.setTextColor(SSD1306_WHITE, SSD1306_BLACK);
  display.setTextSize(1);
  display.setCursor(18, 3);
  display.println("    VOLTMETER    ");
  display.display();
}

float vout = 0.0;
float vin = 0.0;

float R1 = 30000.0;
float R2 = 7500.0;

int value = 0;

void loop() {

  value = analogRead(analogInput);
  vout = (value * 5.0) / 1024.0;
  vin = vout / (R2 / (R1 + R2));

  Serial.print("INPUT Voltage= ");
  Serial.println(vin, 2);

  display.setTextSize(2);
  display.setCursor(20, 15);
  display.println(vin, 2);
  display.setCursor(80, 15);
  display.println("V");
  display.display();

  delay(1000);
}

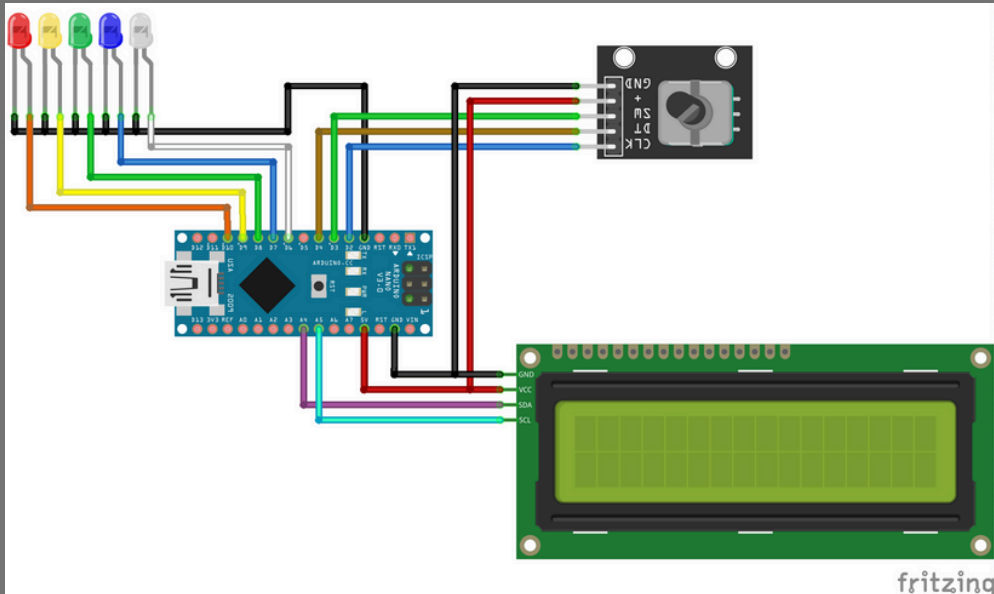
```

• Arduino Rotary Encoder controlled LEDs

• Components :

1. ARDUINO NANO/UNO
2. 16x2 I2C LCD DISPLAY
3. ROTARY ENCODER
4. LED
5. BREADBOARD
6. JUMPERS

• Circuit Diagram :



fritzing

• Main Code :

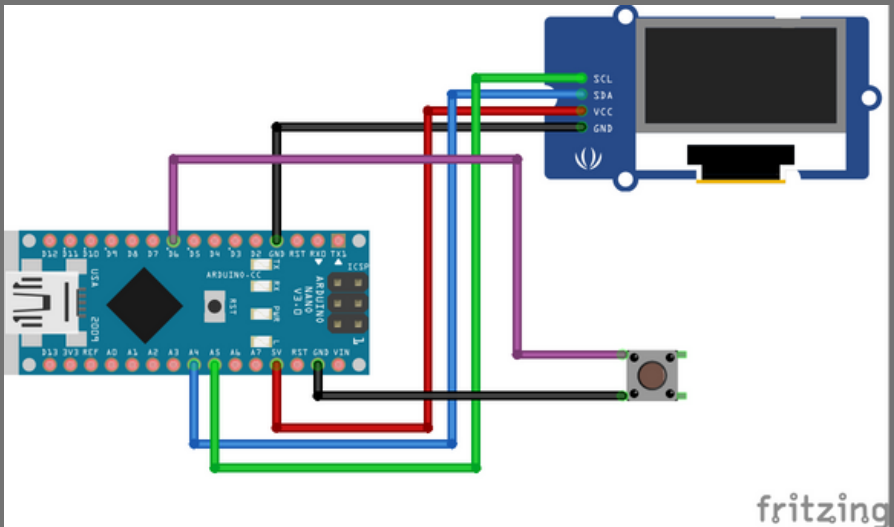
Code Download Link - [click here](#)

- **Arduino OLED Stopwatch**

- **Components :**

1. ARDUINO UNO/NANO/PRO-MINI
2. OLED DISPLAY
3. PUSH BUTTON
4. BREADBOARD
5. WIRES

- **Circuit Diagram :**



- **Main Code :**

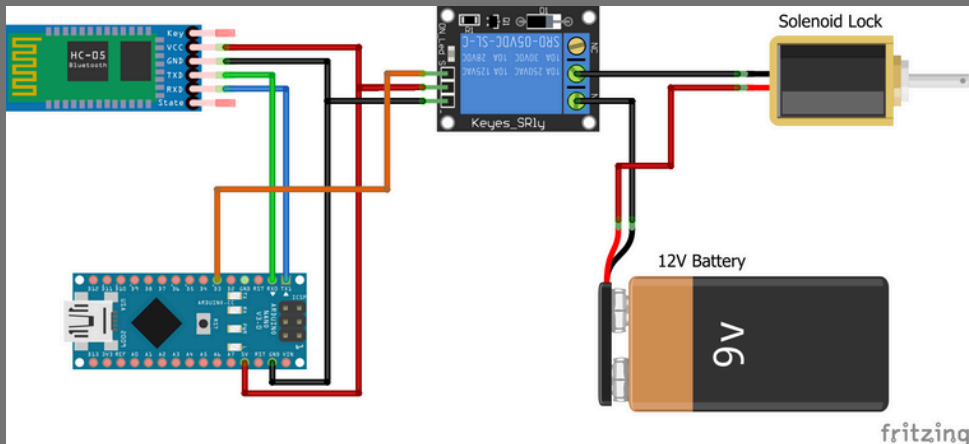
Code Download Link - [click here](#)

- **Arduino BT Door Lock Control**

- **Components :**

1. ARDUINO NANO
2. HC-05 BLUETOOTH MODULE
3. RELAY MODULE
4. BREADBOARD

- **Circuit Diagram :**



- **Main Code :**

```
#define relay1 3 //Connect relay1 to pin D3

void setup()
{
  Serial.begin(9600); //Set rate for communicating with phone
  pinMode(relay1, OUTPUT); //Set relay1 as an output
  digitalWrite(relay1, LOW); //Switch relay1 off
}

void loop()
{
  while(Serial.available()) //Check if there are available bytes to read
  {
    delay(10); //Delay to make it stable
    char c = Serial.read(); //Conduct a serial read
    if (c == '#'){
      break; //Stop the loop once # is detected after a
    }
    word
  }
  readString += c; //Means readString = readString + c
}

if (readString.length() >0)
{
  Serial.println(readString);

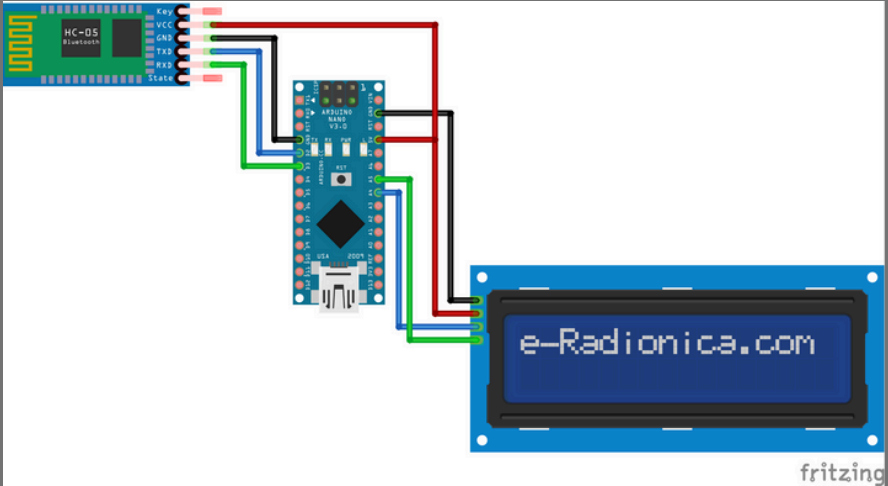
  if(readString == "f success"){
    digitalWrite(relay1, HIGH);
    delay(3000);
    digitalWrite(relay1, LOW);
  }
  readString="";
}
}
```

• Arduino LCD Notice Board

• Components :

1. ARDUINO UNO/NANO
2. 16X2 I2C LCD DISPLAY
3. HC-05 BLUETOOTH MODULE
4. BREADBOARD

• Circuit Diagram :



• Main Code :

goto next page !

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x3C,16,2); //Change the HEX address

#include <SoftwareSerial.h>
SoftwareSerial mySerial (2, 3); // (RX, TX);

String val = "No Data";
String oldval;
String newval = "No Data";
int i = 0;

void setup()
{
    // put your setup code here, to run once:
    lcd.begin();
    mySerial.begin(9600);
    Serial.begin(9600);
    lcd.setCursor(0, 0);
    lcd.print("    Digital    ");
    lcd.setCursor(0, 1);
    lcd.print(" Notice Board ");
    delay(3000);
    lcd.clear();
}

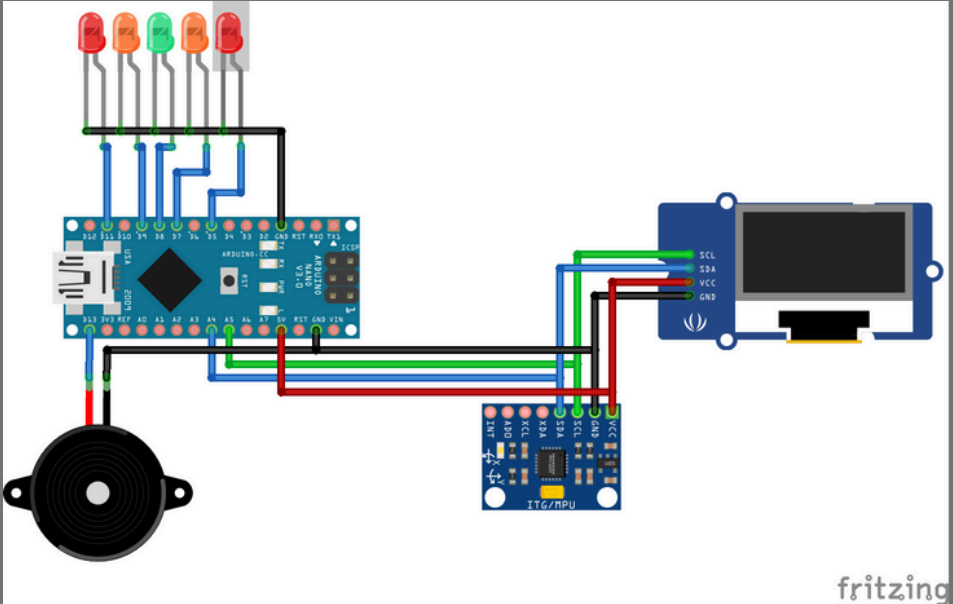
void loop()
{
    val = mySerial.readString();
    val.trim();
    Serial.println(val);
    if(val != oldval)
    {
        newval = val;
    }
    val = oldval;
    Serial.println(val);
    lcd.clear();
    lcd.setCursor(16, 1);
    lcd.print(newval);
    lcd.setCursor(16, 0);
    lcd.print("Notice:");
    for(int counter = 0; counter < 24; counter++)
    {
        lcd.scrollDisplayLeft();
        delay(500);
    }
}
```

• Arduino MPU6050 Digital Spirit Level Measuring Device

• Components :

1. ARDUINO NANO
2. MPU6050
3. OLED DISPLAY
4. BUZZER
5. LED
6. BREADBOARD

• Circuit Diagram :



• Main Code :

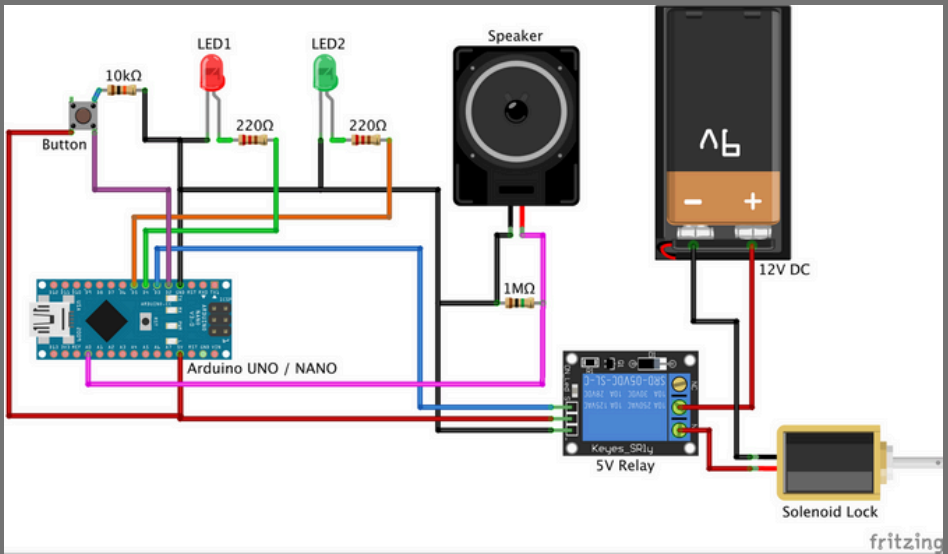
Code Download Link - [click here](#)

• Arduino Secret Knock Pattern Door Lock

• Components :

1. ARDUINO UNO / NANO
2. RELAY MODULE 5V
3. SOLENOID LOCK
4. SPEAKER
5. LED
6. RESISTOR 220, 10K, 1M
7. PUSH BUTTON
8. WIRES
9. BREADBOARD

• Circuit Diagram :



• Main Code :

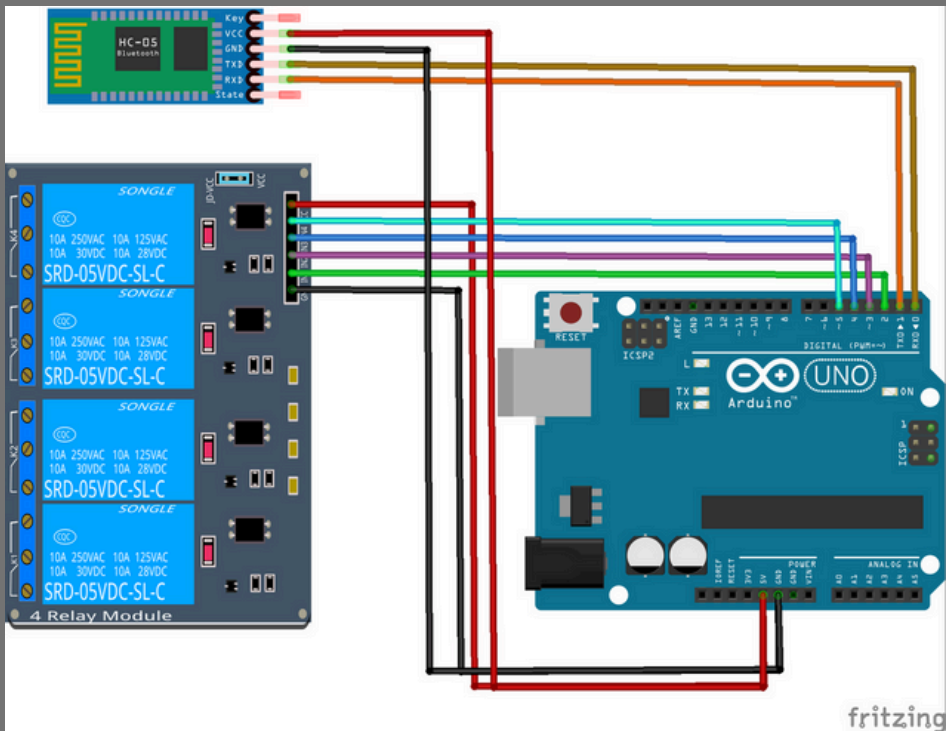
Code Download Link - [click here](#)

• Arduino Home Automation using Bluetooth

- **Components :**

1. ARDUINO NANO
2. HC-05 BLUETOOTH MODULE
3. 4CHANNEL RELAY MODULE
4. BREADBOARD

- **Circuit Diagram :**



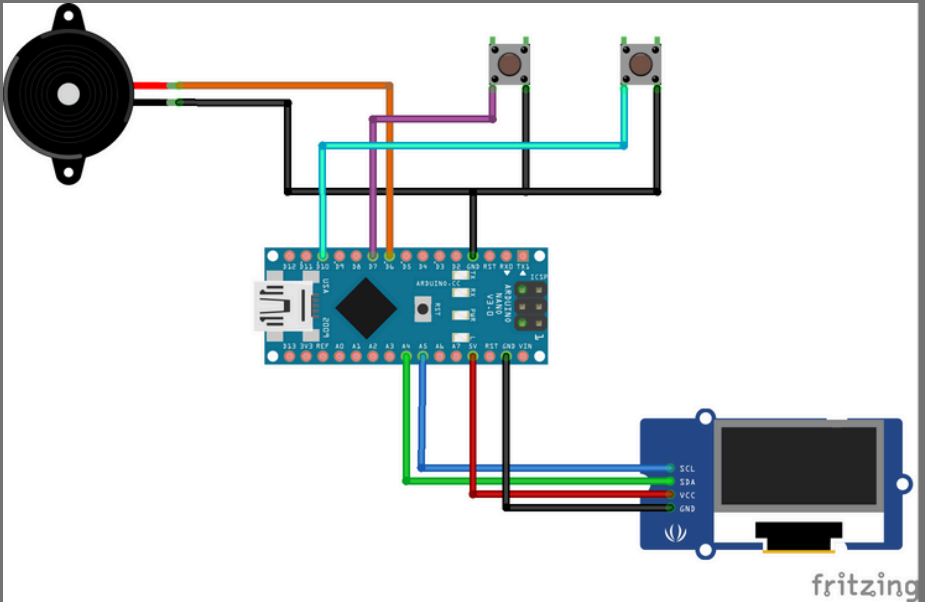
- **Main Code :**

• Arduino Shake DICE OLED Display

• Components :

1. ARDUINO UNO/NANO
2. OLED DISPLAY
3. PUSH TACTILE BUTTONS
4. BUZZER
5. VIBRATION SENSOR
6. BREADBOARD
7. WIRES

• Circuit Diagram :



• Main Code :

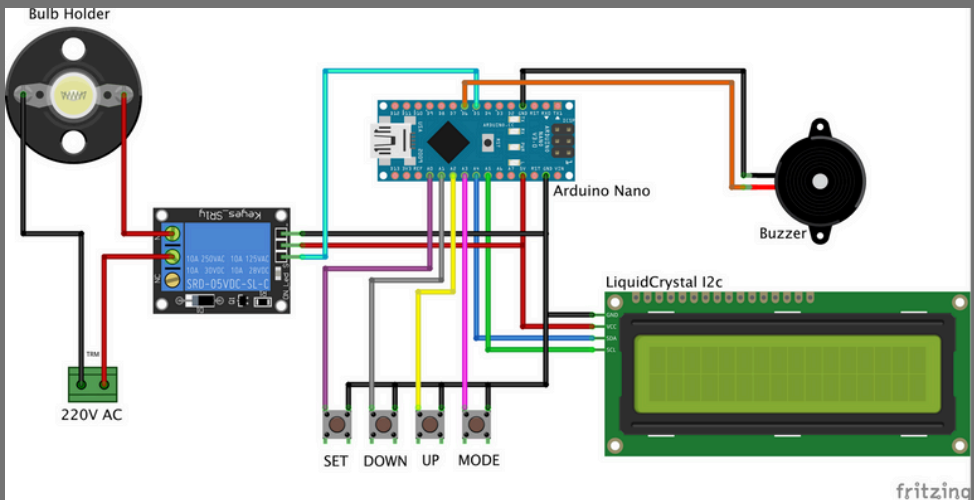
Code Download Link - [click here](#)

• Arduino Timer Control Relay

• Components :

1. ARDUINO NANO/UNO
2. LIQUIDCRYSTAL I2C DISPLAY
3. RELAY MODULE
4. BUZZER
5. PUSH BUTTONS
6. BREADBOARD
7. BULB HOLDER SOCKET
8. WIRES

• Circuit Diagram :



• Main Code :

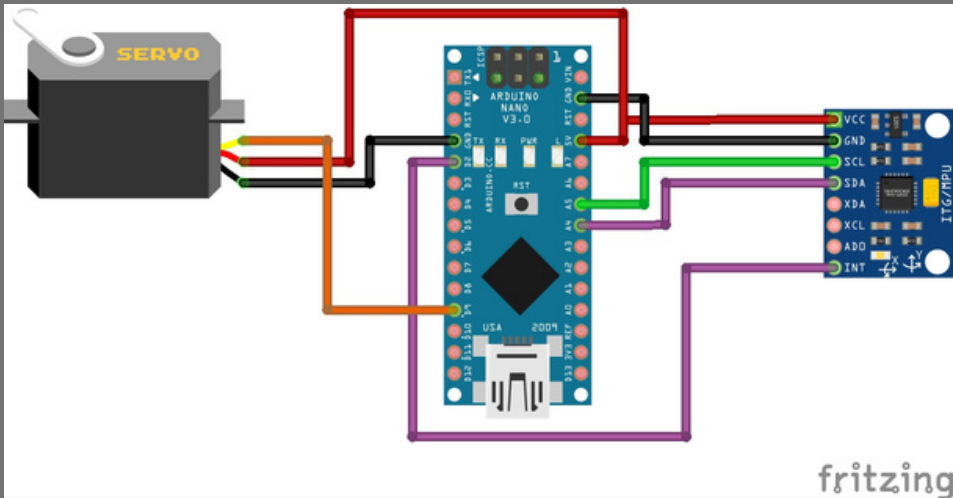
Code Download Link - [click here](#)

• Arduino control Servo with MPU6050

• Components :

1. ARDUINO
2. MPU6050
3. SERVO
4. BREADBOARD
5. JUMPERS

• Circuit Diagram :



• Main Code :

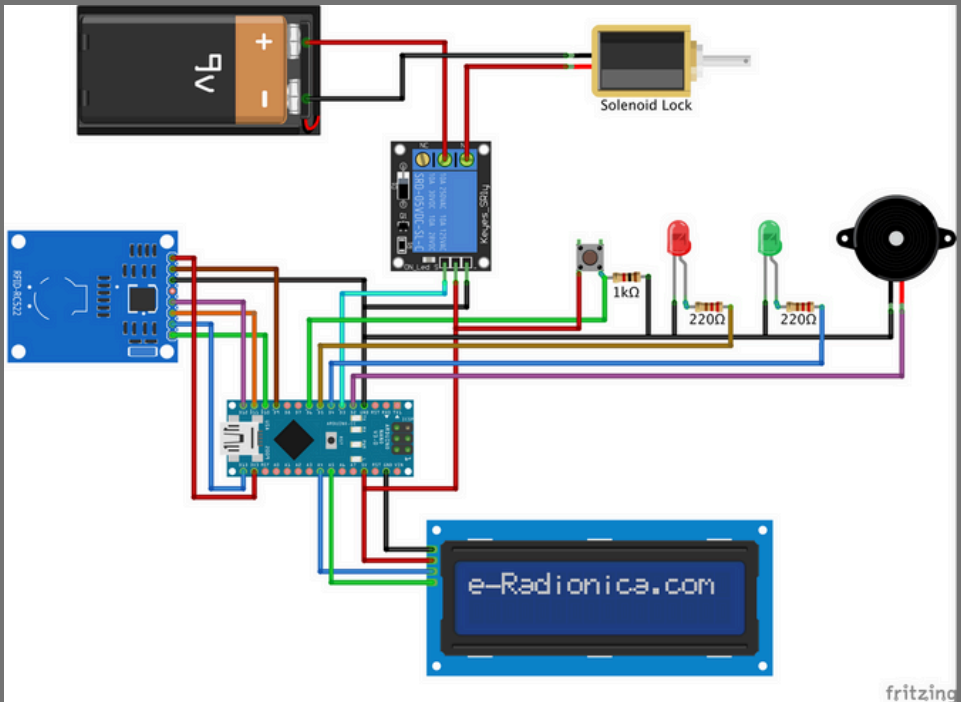
Code Download Link - [click here](#)

- **Arduino RFID Door Lock System with LCD Display**

- **Components :**

1. ARDUINO UNO
2. RFID SENSOR
3. SOLENOID LOCK
4. LED
5. BUZZER
6. BREADBOARD
7. RELAY MODULE
8. PUSH BUTTON
9. BREADBOARD

- **Circuit Diagram :**



- **Main Code :**

Code Download Link - [click here](#)

**Click here to get 24+ more
Arduino Projects Code & Circuits -**

Click Here

Thank You for Exploring Our Arduino Projects eBook!

Dear Reader,

First and foremost, we extend our sincerest gratitude to you for choosing to embark on this journey with us through the world of Arduino projects. Your interest and enthusiasm mean the world to us, and we hope that the experiences within these pages bring you as much joy, learning, and inspiration as they have brought us.

Creating this eBook has been a labor of love, fueled by our passion for electronics, creativity, and the boundless possibilities of Arduino. Every project, tutorial, and piece of advice shared here is crafted with the intention of empowering you to explore, innovate, and create with confidence.

We want to express our gratitude to the Arduino community, whose collaborative spirit and dedication continue to drive innovation and foster learning worldwide. Without the contributions of countless individuals and organizations, this eBook would not have been possible.

To our families, friends, and supporters, thank you for your unwavering encouragement and belief in our endeavors. Your support fuels our passion and propels us forward, even in the face of challenges.

As you delve into the projects contained within these digital pages, remember that creativity knows no bounds. Feel free to experiment, modify, and personalize each project to suit your unique vision and requirements. Let your imagination soar, and don't be afraid to push the boundaries of what you thought possible with Arduino.

Lastly, we invite you to join us in celebrating the joy of making, learning, and sharing. Whether you're a seasoned Arduino enthusiast or a curious beginner, there's always something new to discover and create together.

Thank you once again for choosing our Arduino projects eBook. May it serve as a valuable companion on your journey of exploration and discovery.

Happy tinkering!

Sincerely,

@Mr_circuits007